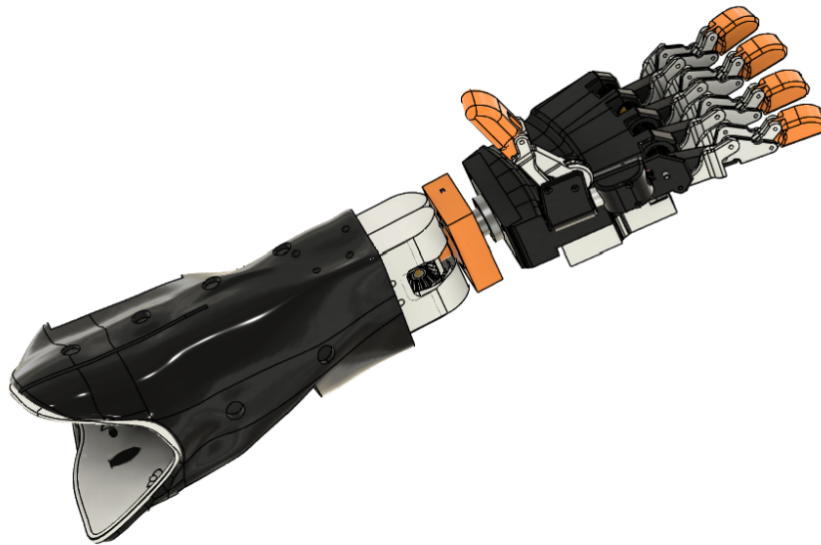


Semester Project Report:

Comprehensive Design of the Safety System of a Prosthesis

N-Pulse — EPFL MAKE



Mathis Finckh

N-Pulse Supervisor: Johann Lugon-Moulin

Academic Supervisor: Prof. A. Schmid

June 5, 2026

Contents

1	Introduction	2
2	State of the Art	2
2.1	Thermal discomfort and socket temperature	2
2.2	Fall and shock detection in wearables	2
2.3	Research gap and project positioning	3
3	Hardware Design	3
3.1	Requirements and sensor selection	3
3.1.1	Microcontroller Unit (MCU)	3
3.1.2	Temperature and Relative Humidity (RH)	4
3.1.3	Temperature	4
3.1.4	Current	5
3.1.5	Acceleration	5
3.2	System architecture	5
3.3	Known hardware limitations	6
3.4	Test Bench	6
4	Firmware Architecture	7
4.1	Design philosophy	7
4.2	Initialization	7
4.3	Main loop	7
4.4	Interrupt handling	8
4.5	Non-blocking temperature reading	8
4.6	Logging	8
4.7	Graphical User Interface	8
5	Engineering Justification	8
5.1	Sensor checking rate	8
5.1.1	Thermal response time	8
5.1.2	Current and Acceleration reaction time	10
5.1.3	Other sensors	10
5.2	Communication buses	11
5.2.1	I ² C	11
5.2.2	1 Wire	12
5.3	Electromagnetic Noise	12
5.3.1	Stall Case	13
5.3.2	Nominal (loaded) Case	13
5.4	Shock Detection Threshold	14
5.5	Redundancy	14
6	Results and Validation	15
7	Future work	15
8	Conclusion	18
	References	19
	Annex	22

1 Introduction

N-Pulse is a student-led EPFL MAKE initiative dedicated to developing open-source biomedical technologies, including a transradial arm prosthesis. A transradial prosthesis replaces the forearm and hand below the elbow, restoring grasping and wrist function through motorized actuation. As the prosthesis becomes more capable, more actuators, higher-capacity batteries, and complex control electronics, the consequences of electrical and thermal faults become more serious, both for the system and for the user.

The current arm includes only a passive fuse for electrical protection. This provides no active monitoring, no diagnostic capability, and no ability to respond to faults before damage occurs. Overheating motors, an overloaded battery, excessive contact temperature at the socket or a hard impact are all scenarios that a fuse alone cannot address.

This project addresses that gap by designing a sensor-based safety system for the prosthetic arm. The system continuously monitors six failure modes: overcurrent, motor and battery overheating, high ambient temperature near the electronics, elevated socket contact temperature, high humidity, and high-impact events. It is designed to cut power in response to those.

The scope of this project covers sensor selection and justification, system architecture, firmware design, and engineering validation of the proposed approach. It does not cover full mechanical integration into the arm or clinical validation. The report is structured as follows: Section 2 reviews the relevant literature, Section 3 presents the hardware design and sensor selection, Section 4 describes the firmware architecture, Section 5 provides engineering justifications for the key design choices and Sections 6, 7, 8 present the results and validation of the prototype test bench, some suggestions for future work directions and a conclusion.

2 State of the Art

2.1 Thermal discomfort and socket temperature

A common issue with prosthetics is discomfort at the socket, where the device attaches to the user, with more than 50% of users affected, particularly with lower limb prosthetic users. This discomfort is largely due to excessive heating [1] as the three main ways the human body regulates heat, radiation, convection and evaporation, are blocked by the socket. This can cause excessive sweating, skin damage, infections and ultimately abandoning the device [2]. The current improvements are passive, with improved materials and ventilation [3].

Socket skin contact temperature is therefore clinically relevant. The IEC 60601-1 medical standard dictates a regulatory limit of 41°C for prolonged contact (up to 43°C with special justification) [4]. Even at relatively low temperatures, the human skin experiences slow burns.

However, real-time electronic monitoring of skin contact temperature is not standard practice in prosthetics [5] and should be a helpful metric to monitor, especially in early design iterations or later real user testing.

2.2 Fall and shock detection in wearables

The literature suggests that normal life activities like walking, running and hopping can generate accelerations at the wrist of up to 10 G's ($\approx 100 \text{ m/s}^2$) [6]. On the other hand, harsh events like a punch [7], or falling on one's outstretched arm [8] generate accelerations of 80-230 G's and 80-130 G's (2.57-4.20 kN over $\approx 20 \text{ ms}$) respectively.

However, differentiating between different high acceleration events requires finer classification approaches. Indeed, fall detection is an active area of research, particularly for elderly people who are at risk of severe consequences from such falls. Many of these methods use machine learning to achieve good classification [9] on accelerometer data, or even vision and more. These methods require labeled data and significant processing. A threshold-based approach is sufficient

for the current goals of the safety system; more sophisticated classification methods remain a possibility for future development.

Fall and shock detection for prosthesis safety and overall usage data collection are rather unexplored in publicly available literature and it is expected to be a relevant metric to monitor.

2.3 Research gap and project positioning

Beyond thermal discomfort, lithium-ion batteries used in powered prosthetics pose a thermal runaway risk that is well-documented in wearable electronics [10]. Real-time current monitoring to detect overcurrent before fuse activation is therefore equally important in prosthetic design. Similarly, DC motor current limitation to prevent overheating has been studied in isolation, but not as part of an integrated prosthetic safety system [11].

Safety monitoring in prosthetics is underexplored, although some individual issues are studied and sometimes even addressed, no integrated real-time safety system was identified in the literature reviewed for this project. The research focus is usually on actuation: control, haptics and interfacing: EMG, EEG. This may be explained by the fact that those areas of research are where the progress needs to be made for these wearable prosthetics to be adopted. The products that are already on the market, although we assume they have safety systems in place to meet the strict regulations around medical devices, do not share the detail of their designs.

This project is meant to fill this gap by implementing an integrated, real-time safety monitoring system for a transradial prosthetic arm.

3 Hardware Design

The most relevant failure risks the prosthetic will face are:

- Overcurrent
- Overheating of the battery and motors
- High ambient temperature near the electronics
- Elevated contact temperature at the socket
- High relative humidity
- High-impact events

It is therefore critical to select hardware that can monitor these variables over the ranges expected in operation.

3.1 Requirements and sensor selection

3.1.1 Microcontroller Unit (MCU)

The safety system requires a microcontroller able to handle two separate I²C buses, dedicated hardware interrupt pins for the current sensor and accelerometer, and sufficient performance for real-time sensor management and future diagnostic expansion. A MicroSD slot for persistent data logging and wireless communication capability are also desirable features.

The Adafruit Metro ESP32-S3 was selected to meet these requirements [12], [13]. It is built around the ESP32-S3 (18.0 x 25.5 x 3.1 mm), which provides dual-core processing, Wi-Fi and Bluetooth LE wireless connectivity, and a large number of configurable GPIO pins. It natively supports multiple I²C buses and 1-Wire-compatible communication. A built-in MicroSD slot supports future data logging once the corresponding module is integrated. For prototyping, STEMMA QT connectors allow direct connection of sensors without soldering. In active use

the board draws 60-80 mA, and it supports a deep sleep mode at approximately 100 μ A, leaving room for future low-power operation.

3.1.2 Temperature and Relative Humidity (RH)

The ambient air temperature requirements vary from a maximum of 125 °C, down to 65°C for certain electronic components (e.g. the MCU [13]). Therefore, the conservative limit of 65°C for ambient air temperature is chosen. The RH will vary greatly depending on the day and may reach 100%. It is also outside direct control, but remains relevant to monitor, as it may help diagnose certain fault conditions.

The SHT41 was chosen for inside the prosthetic, near the electronics, as it measures relative humidity up to 100% and temperature up to 125°C with sufficient precision (± 1.0 %RH, $\Delta T = \pm 0.1$ °C in the range of interest). It has a very low power consumption (average 0.4 μ A active measurements, 80 nA sleep). It is very small (1.5 x 1.5 x 0.5 mm) for further implementation on the prosthesis (e.g. custom Printed Circuit Board (PCB)). It communicates using the I²C bus which makes it simple to use and prototype with. It was chosen with the breakout board from Adafruit which pre-implements all the needed support components and STEMMA QT connectors for ease of prototyping. [14], [15]

Additionally, the Adafruit Si7021 was chosen for the socket as it has almost identical properties (3 x 3 x 0.75 mm) with an additional protective PTFE filter. Being more exposed to the outside environment, the filter should prevent dust and water to clog the sensor and ensure accurate sensing longer. [16], [17]

3.1.3 Temperature

For the battery, socket and motor temperature, a very compact sensor is needed to put in direct contact with those components and be able to read values close to their actual temperature through conduction. The three component monitored this way are :

- **Battery :** The INR18650-35E battery cells are rated up to 60°C during discharge (usage of the arm). [18]
- **Socket :** In accordance with IEC 60601-1, the contact temperature must remain below 43°C for prolonged contact, and values above 41°C require justification. The socket contact temperature with the patient's arm will therefore be limited to 41°C to prevent slow burns. [4], [19]
- **Motors :** The motor casings are rated for temperatures from -10 to 85°C [20].

The DS18B20 temperature sensor was chosen as it covers the whole range of potentially observable temperatures (-55 to 125°C) with sufficient accuracy (± 0.5 °C Accuracy from -10°C to +85°C). It is sufficiently compact for our use (5 x 5 x 3.6 mm, e.g. motor diameter = 16 mm). It communicates using the 1-Wire bus which requires only one data port pin per bus line. It is a digital sensor with internal temperature conversion, which simplifies the implementation by avoiding external calibration and reducing the number of required pins compared to a simple thermistor. Its main limitation is the conversion time which varies from 90-750 ms depending on the resolution. [21]

With the DS18B20, a 1-Wire to I²C converter is needed to simplify the wiring from the hand to the forearm. For that purpose, the Adafruit DS2484 breakout board was selected (2 x 3 mm). [22], [23]

3.1.4 Current

Battery current must be monitored in real time to be able to cut power before excessive damage occurs in the on-board electronics or the main fuse needs to be replaced. The fuse should remain the actual last resort safety component. The current sensor should therefore protect the system by triggering a hardware interrupt on overcurrent (short circuits or abnormal power draw). Theoretically the maximum power draw from the 8 motors stalling is 5.3 A. From the datasheet, with a stall torque of $M_H = 6.5$ mNm, a torque constant of $k_M = 9.96$ mNm/A, the stall current per motor is $I_{stall} = M_H/k_M = 0.653$ A [20]. A large margin is kept on the maximum measurable current, to account for the rest of the system drawing current simultaneously and potentially later iterations of the prosthesis having more degrees of freedom (more actuators).

The chosen sensor is the Adafruit INA260 Voltage, Current and Power Sensor. It has a current sensing resistance of $2\text{ m}\Omega$, it can handle 15 A continuously with high accuracy (0.15% gain error, maximum 5 mA offset) and voltages from 0-36 V (14.8 V for our current battery). It draws only $310\text{ }\mu\text{A}$ and has a programmable alert pin which will allow for a hardware interrupt for fast response time. Again, the sensor itself is small enough for later PCB implementation (5×4.4 mm) it communicates via I²C with 16 programmable addresses. Finally, it has STEMMA QT connectors on the breakout board. [24], [25]

This sensor will be able to safely monitor the battery current output, even during high current failure cases. It has a wide enough sensing and operating range to be compatible with future changes like a battery voltage increase or an increased number of actuators (higher potential maximum current).

3.1.5 Acceleration

One of N-pulse’s goals is to design the prosthetic for it to withstand a 2 meter drop, but for the time being, it is largely 3D-printed and likely vulnerable to impact. Even in the hypothetical case where it could survive that fall, its integrity would most likely be compromised if the patient were to fall on it with outstretched arm. There is therefore a need to monitor accelerations (shocks) and be able to detect transient high-g events. Normal daily activities are expected to remain below approximately 10 g (or 981 m/s^2), whereas actual falls should be 80 g’s and above (see section 5.4 for more details).

Given this ballpark, the Adafruit LIS3DH triple-axis accelerometer ($3 \times 3 \times 1$ mm) was selected for its ± 16 g sensing range (and 10000 g shock survival). It also has a programmable hardware interrupt for fast response time. It draws at most $11\text{ }\mu\text{A}$, communicates through I²C (with two possible addresses) and has STEMMA QT connectors on the breakout board. [26], [27]

This sensing range was chosen to distinguish between normal activities and safety compromising events.

3.2 System architecture

Figure 1 illustrates the system architecture, showing the spatial distribution of components, sensors, and wiring across the prosthesis.

In the hand, there are 6 motors using a common electronic speed controller (ESC). Thermal sensors are distributed alternately across the motors on two 1-Wire buses for redundancy, each managed by a DS2484 adapter and transferred to two I²C lines. An additional combined temperature and relative humidity sensor (SHT41) monitors ambient conditions inside the hand casing. The hand communicates with the forearm through two I²C buses routed across the actuated wrist joint.

The forearm of the prosthesis contains two more motors for the wrist actuation, which are both equipped with thermal sensors. It also contains the battery, and the rest of the electronics (ESC, power board, main MCU). A 1-Wire thermal sensor is mounted in direct contact with the

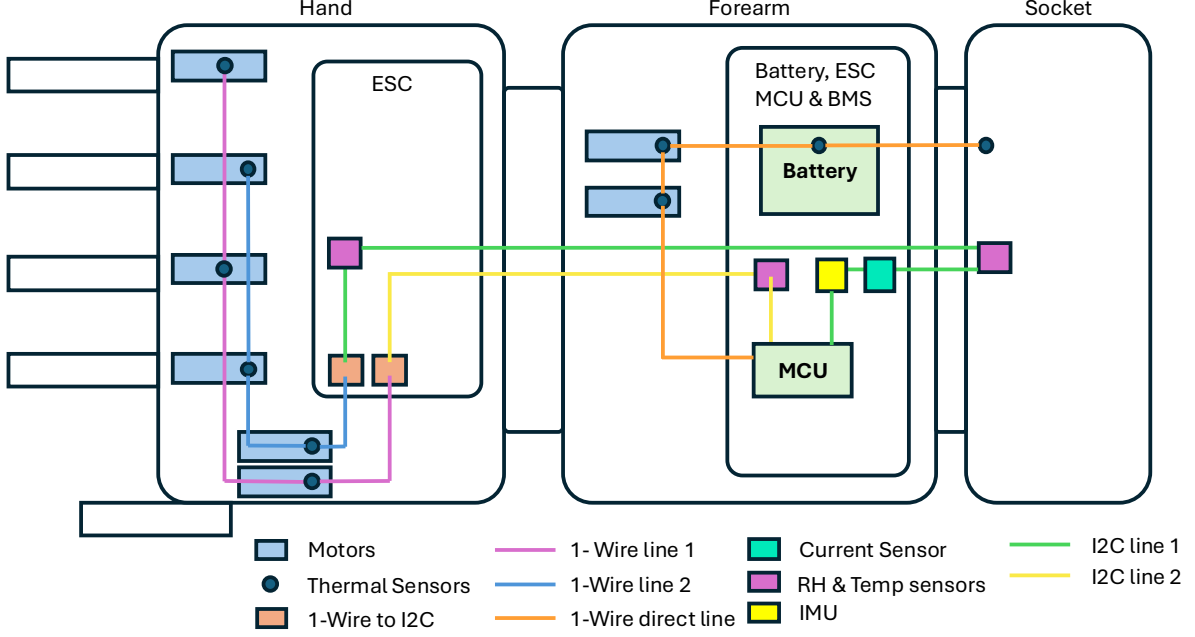


Figure 1: Prosthesis Safety System Architecture Schematic

battery to monitor cell temperature. The safety system MCU, the accelerometer, the current sensor and an additional RH and temperature sensor are also located in the forearm. The 1-Wire thermal sensors (including the one in the socket) are directly wired to the MCU and the other sensors are distributed between the two I²C lines to balance bus load and maintain redundancy.

The socket has a final RH and temperature sensor as well as a 1-Wire thermal sensor. Although the socket is physically part of the forearm assembly in a transradial prosthesis, it is treated as a distinct sub-system here, as its sensing role (skin-contact temperature and external humidity) differs from that of the forearm electronics compartment.

3.3 Known hardware limitations

The electronically controlled power switch and the MicroSD module, both part of the original design, were not delivered in time for this prototype. As a result, two planned features are not implemented in the current version: data logging of operational metrics over extended testing periods, and software-controlled battery disconnection by the safety MCU. These remain important capabilities for future iterations, as they would complete the autonomous fault response and long-term diagnostics functions of the safety system.

3.4 Test Bench

A test bench demonstrating all that has been presented in this report was built. It was assembled on a wood base, arranged in sub-areas : hand, forearm and socket. Each "area" on a separate perfboard, so that the wiring can be hidden and the system feels more "plug and play". The hand area connects to the forearm area using two STEMMA QT cables, the socket area connects the forearm area using a custom cable with JST connectors (custom made for ease of use for the next users, no wrong wiring) and the forearm area connects to the MCU with two custom cables (JST connectors to male header pin).

The acceleration, current, RH and temperature thresholds have been lowered compared to the discussed values, for demonstration purposes. A potentiometer was added to demonstrate

the current changing, the alert and interrupt working live (without burning a battery). A 3D printed motor mount was placed in front of two thermal sensors to mimic their final use.

4 Firmware Architecture

4.1 Design philosophy

The firmware is structured around two parallel concerns: fast fault protection and periodic sensor monitoring. Critical faults (overcurrent and shock) are handled through hardware interrupts for minimal reaction time, while slower environmental metrics are evaluated at a fixed rate for logging and threshold checking. This separation ensures that the protective response is never delayed by routine sensing tasks.

The overall execution flow of the code is summarized with the following pseudo-code :

```
Initialize buses, sensors and interrupts

If interrupt pin triggered
    Set temporary flag to True
    Cut power

Loop:
    check interrupt-based faults
    If interrupt flag set (overcurrent or shock):
        read I2C status register, clear latch

    Every 500 ms:
        read all sensors
        evaluate thresholds
        if critical threshold exceeded: request power cut
        log data to CSV
        Trigger non-blocking DS18B20 temperature conversion
```

The whole code is available in the annex 8.

4.2 Initialization

At startup, all constants, I²C addresses, safety thresholds, and fault-relevant metrics are defined. The two I²C buses as well as the direct 1-Wire bus are initiated. All sensors on the buses are identified, and those requiring configuration are set up: resolution and sampling frequency for the DS18B20 thermal sensors, output data rate and sensing range for the accelerometer, and alert thresholds for the hardware interrupt outputs of both the accelerometer and the current sensor.

4.3 Main loop

The main loop runs without artificial delay to remain responsive to hardware interrupts at all times. Two mechanisms operate in parallel:

- **Interrupt-based fast fault detection:** At each iteration, the hardware interrupt flags for the current sensor and the accelerometer are checked. If either flag is set (and therefore power is being cut), a confirmation and latch clearing is done via I²C . A software flag allows the power cut to be disabled for testing and demonstration purposes; in that case, the latch clearing and status read remain useful for diagnosing the triggering condition.

- **Periodic sensor monitoring:** At 2 Hz (see section 5.1), all sensors are read and evaluated against their respective thresholds. If any metric exceeds a critical limit, a power cut may also be requested. All values are logged to the MicroSD card and printed to the serial monitor in CSV format.

4.4 Interrupt handling

The current sensor and accelerometer have a direct wired connection to the MCU (dedicated GPIO pin) for a rapid interruption in cases of exceeded limits. This immediately sets a flag, and requests a power cut. This is later confirmed in the next loop iteration, reading the I²C status registers and clearing the sensor latch. This ensures minimal reaction time.

4.5 Non-blocking temperature reading

Since the temperature readings with the DS18B20 take about 94 ms (with 9 bit resolution), the conversion is started in advance and stored, so that the later sensor reading can happen without that delay. This is done to avoid stalling the main loop, so it can stay responsive throughout the conversion time.

4.6 Logging

The logging and serial printing is done in a predefined CSV (Comma Separated Value) format. This format supports both live visualization and later log data analysis. The format is the following :

```
t_ms,
hand_temp_c, hand_rh, wrist_temp_c, wrist_rh, socket_temp_c, socket_rh,
bus0_ds1_c, bus0_ds2_c, bus0_ds3_c, bus1_ds1_c, bus1_ds2_c, bus1_ds3_c,
battery, socket, direct_ds1_c, imu_ax_g, imu_ay_g, imu_az_g, imu_mag_g,
current_ma, bus_mv, power_mw, temp_limit_flag, humidity_limit_flag,
overcurrent_limit_flag, shock_flag, power_cut_flag,
sensors_ok
```

4.7 Graphical User Interface

The graphical user interface (GUI) is a separate code that runs on a local computer connected with a cable to the MCU. It is programmed using python and the Plotly and Dash libraries. It periodically reads the CSV lines received using the serial feed and plots them live. It is possible to tick off (stop plotting) some sensors as some plots get crowded. Additionally, the GUI displays the different flags, as well as the power cut flag a general sensor check.

5 Engineering Justification

5.1 Sensor checking rate

The rate at which we check the sensors could be a limitation in detecting faults in time. To address that, this next section will explore the different failure detection times.

5.1.1 Thermal response time

For the motors, given a nominal voltage of 12 V and an electrical resistance of 18.2 Ω , the stall current and power dissipated as heat are :

$$I_{stall} = \frac{U_N}{R} = \frac{12}{18.2} = 0.659 \text{ A}$$

$$P_{stall} = I_{stall}^2 \cdot R = \frac{U_N^2}{R} = 12^2 / 18.2 = 7.912 \text{ W}$$

The relevant thermal parameters from the motor datasheet are:

- Thermal resistance, winding : $R_{th1} = 16 \text{ K/W}$ (Datasheet)
- Thermal resistance, casing with ambient air (metal) : $R_{th2,m} = 6.1 \text{ K/W}$
- Thermal time constant, winding : $\tau_{w1} = 11 \text{ s}$ (Datasheet)
- Thermal time constant, casing : $\tau_{w2,m} = 25 \text{ s}$ (Datasheet)

Steady state winding temp increase :

$$T_{winding,ss} = T_{amb} + P \cdot (R_{th1} + R_{th2}) = 22 + 7.912 \cdot (16 + 6.1) = 196.855 \text{ }^\circ\text{C}$$

$$T_{casing,ss} = T_{amb} + P \cdot R_{th2} = 22 + 7.912 \cdot 6.1 = 70.263 \text{ }^\circ\text{C}$$

First-Order transient response for the casing:

$$T(t) = T_{ss} - (T_{ss} - T_{amb}) \cdot e^{-\frac{t}{\tau}} = 70.263 - 48.263 \cdot e^{-\frac{t}{25}}$$

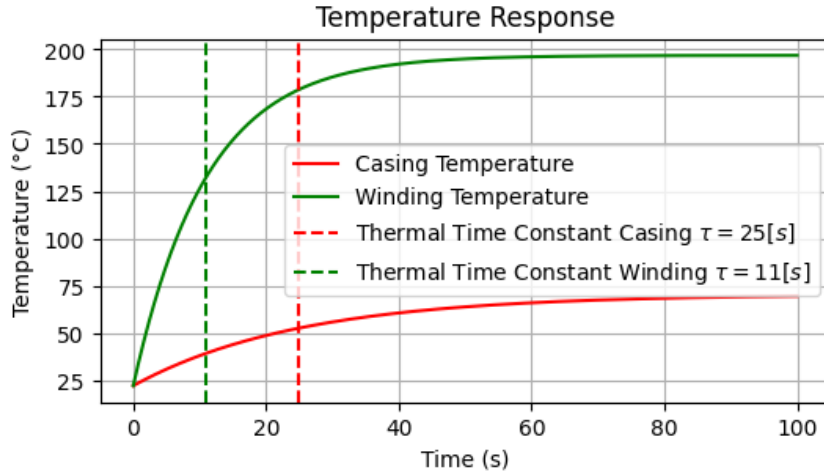


Figure 2: Thermal response of the casing at stall (with thermal time constant : time until temperature motor casing/winding reaches 63% of steady-state temperature)

At stall, the time to reach the maximum permissible winding temperature of $100 \text{ }^\circ\text{C}$ is :

$$t_{maxtemp} = -\ln \left(\frac{T_{winding,ss} - 100}{T_{winding,ss} - T_{amb}} \right) \cdot \tau_{w1} = 6.498 \text{ s}$$

The maximum theoretical temperature the casing would achieve at ambient temperature of $22 \text{ }^\circ\text{C}$ under stalling conditions is $70 \text{ }^\circ\text{C}$. This is below the maximum allowable temperature for the casing. However, the winding temperature response time is much shorter and it will reach temperatures higher than its allowable maximum without it being detected through the casing. Under stalling conditions, the winding would reach the maximum permissible temperature of $100 \text{ }^\circ\text{C}$ within 6.5 s, at which point the casing temperature would only be $33 \text{ }^\circ\text{C}$. This means that measuring casing temperature cannot be a hardware protection measure, but a user and structure (3D printed PET-G) protection measure. This is partly explained by the fact that there is usually no direct contact between the wiring and the casing and that layer of air acts as thermal isolation between the wiring and the casing.

For this purpose, the casing thermal response time $\tau = 25$ s is slow enough for it not to be a limitation in our choice of sensor reading frequency. For a slowly evolving signal like this, any threshold crossing can be detected with sufficient resolution if the sampling period is much smaller than the characteristic timescale of variation. Applying an approximation of the Nyquist-Shannon criterion as a conservative, stricter bound, where the minimum sampling frequency must exceed twice the inverse of the characteristic timescale, the sensing would require a minimum sampling frequency of : $f_{min} = \frac{2}{\tau} = \frac{2}{25} = 0.08$ Hz.

A similar argument can be made for ambient air temperature near electronics: it evolves even more slowly and is therefore not a limiting factor for the sampling rate.

Concerning the battery heating, the datasheet does not provide exact numbers, so this analysis relies on a general order-of-magnitude argument. Battery temperature varies on the scale of minutes under normal conditions. Even under fault conditions such as high-current internal battery shorts (for Lithium-Ion), temperature rises of 20 °C take over 10 seconds [28], a highly unlikely scenario over which the system has limited direct control. This represents the most extreme case in terms of battery heating, and treating 10 s as the characteristic timescale gives a minimum detection frequency of $f_{min} = 0.1$ Hz.

5.1.2 Current and Acceleration reaction time

The current and acceleration are monitored using interrupt on dedicated pins for fast reaction time, so the sensing frequency is not the limiting factor for these metrics.

The main loop executes in approximately 5–15 μ s per iteration on the ESP32-S3 at 240 MHz, giving a worst-case time of one loop iteration before interrupt handling (flag-check) begins. For the over-current, the total reaction time is approximately 1.3 ms, dominated by the current sensors 1.1 ms ADC conversion window. For the shock detection, the total reaction time is approximately 5 ms, dominated by the accelerometer duration filter of 1 threshold-exceeding samples at 200 Hz.

Both latencies are negligible relative to the damage timescales they protect against.

In the event of a hard short circuit, current rises on a microsecond timescale, beyond the reach of any software-based protection. This case is handled by the hardware fuse, which acts as the last-resort analog protection. The software interrupt path targets a different failure mode: sustained overcurrent arising from motor stalls, mechanical jams, or abnormal power draw. In these cases, the resulting damage is thermal in nature, developing over tens of milliseconds to seconds [29][30]. The estimated interrupt reaction time of 1.3 ms is therefore well within the required response window for these fault conditions.

A fall event requiring power cut is not millisecond sensitive, it simply means that the arm's integrity is compromised and it should not be used further.

5.1.3 Other sensors

The additional tracked metric is humidity. Our system has no control over humidity and it is a slow evolving metric, as it has to do with ambient air, so the tracking of it follows the same slow evolving requirements as temperature checking. Furthermore, general electronics can operate at high levels of RH, but it is recommended not to do so for long periods of time. Nevertheless, it is still something relevant to monitor, especially in cases of failures like shorts.

For logging and bookkeeping needs, a sensing and logging frequency of 2 Hz was chosen, a margin of $2 \text{ Hz} / 0.1 \text{ Hz} = 20\times$ above the most demanding bound.

5.2 Communication buses

5.2.1 I²C

To accurately evaluate the I²C bus load and ensure signal integrity, the total bus capacitance (C_b) must be established. The capacitance budgets in tables 1 and 2 are calculated using the following conservative baseline estimates:

- **Devices:** 10 pF per device (MCU and sensors), which takes into account the negligible capacitance of the commercial breakout boards' micro-traces [31].
- **Connectors:** 3 pF, accounting for both JST-SH cable plugs and header pins [32].
- **Cables:** 50 pF/m, based on the physical properties of the unshielded 28 AWG wire used in the STEMMA QT cables [33].
- **Perfboard:** 1.0 pF/cm, representing the stray trace capacitance (significant overestimate since no ground plane) [31].

Source	Qty / Length	Contribution (pF)
IC Pins	5 devices	50
Connectors	8 units	24
Cables	0.4 meters	20
Perfboard	10 cm	10
Total C_b		104

Table 1: Capacitance budget for I²C Line 1 ($R_p = 2.5 \text{ k}\Omega$, $C_{b,max} = 141.6 \text{ pF}$).

Source	Qty / Length	Contribution (pF)
IC Pins	4 devices	40
Connectors	8 units	24
Cables	0.6 meters	30
Perfboard	10 cm	10
Total C_b		104

Table 2: Capacitance budget for I²C Line 2 ($R_p = 3.33 \text{ k}\Omega$, $C_{b,max} = 106.2 \text{ pF}$).

While these values are well within the 400 pF absolute maximum bound for 400 kHz Fast Mode I²C, the true limit of the system is constrained by the onboard hardware. Each sensor breakout board features an integrated 10 $k\Omega$ pullup resistor. When connected to the same bus, these resistors act in parallel, resulting in an equivalent line resistance (R_p) of 3.33 and 2.5 $k\Omega$ for the 3- and 4-component buses, respectively.

$$C_b < \frac{t_{rise}}{0.8473 \cdot R_p} \quad (1)$$

The I²C Fast Mode specification mandates a maximum rise time (t_r) of 300 ns [31]. Based on equation (1), the maximum allowable bus capacitance dictated by these specific equivalent resistances is tightly constrained to 106.2 pF and 141.6 pF. Both the lines are estimated to fall below that, especially considering they were computed using conservative approximations.

Also note that I²C systems have a minimum pullup resistance requirement. With a $V_{cc} = 3.3V$, $V_{ol} = 0.4 \text{ V}$ (maximum low-level output voltage at $I_{OL} = 3 \text{ mA}$, for Fast Mode specification), we have:

$$R_{min} = \frac{V_{cc} - V_{ol}}{I_{ol}} = 967 \text{ }\Omega$$

However, the system is far from this limit. Adding I²C devices with built in pullup resistors effectively both increases the bus capacitance and lowers the total resistance. This setup could theoretically take up to 10 devices (each with 10 kΩ pullup resistors in the case of Adafruit breakout boards) before hitting the minimum resistance requirement.

$$R_{equ} = \frac{1}{N \cdot \frac{1}{10 \text{ k}\Omega}} = \frac{10 \text{ k}\Omega}{N} \Rightarrow N < \frac{10 \text{ k}\Omega}{R_{min}} = 10.34$$

On a practical note, the prototype never had issues communicating with I²C , which confirms these calculations.

5.2.2 1 Wire

In the datasheet for the DS18B20 thermal sensor, the "1-Wire bus requires an external pullup resistor of approximately 5 kΩ" (4.7 kΩ in the schematics) [21]. The same was therefore used for the direct 1-Wire line. The 1-Wire buses connected to the I²C to 1-Wire converter (DS2484) did not require an additional pullup resistor, as the DS2484 integrates an active pull-up on its 1-Wire port [22].

5.3 Electromagnetic Noise

There are two main effects contributing to the noise through the cables or crosstalk : the capacitive and inductive coupling

$$V_{noise} = V_{cap} + V_{ind}$$

with

$$V_{cap} = C_m \cdot R_{pull} \cdot \frac{V_{spike}}{\tau_{comm}}$$

$$V_{ind} = L_m \cdot \frac{I}{\tau_{comm}}$$

and R_{pull} the I²C total pullup resistance.

As a representative geometry, assume the I²C and motor cables are running parallel d = 5 mm from one another over l = 50 mm, with cable radius r = 0.2 mm ($\epsilon_0 = 8.854 \cdot 10^{-12}$ and $\mu_0 = 4\pi \cdot 10^{-7}$ H/m).

Mutual capacitance [34]

$$C_m = \frac{l \cdot \pi \cdot \epsilon_0}{\ln\left(\frac{d}{r}\right)} = 0.432 \text{ pF}$$

Mutual inductance

$$L_m = \frac{\mu_0 \cdot l}{\pi} \cdot \ln\left(\frac{d-r}{r}\right) = 63.56 \text{ nH}$$

Commutation Time Constant

$$\tau_{comm} = \frac{L_{motor}}{R_{motor}} = \frac{498 \mu H}{18.2} = 27.67 \mu s$$

5.3.1 Stall Case

This is the case where the motor is stopped still by a strong counter torque, so the current is maximum.

$$V_{spike} = L_{motor} \cdot \frac{I_{stall}}{\tau_{comm}}$$

since $\tau_{comm} = L_{motor}/R_{motor}$, this simplifies to :

$$V_{spike} = I_{stall} \cdot R_{motor} = V_N = 12 \text{ V}$$

$$V_{cap,stall} = C_m \cdot R_{pull} \cdot \frac{V_{spike,stall}}{\tau_{comm}} = 0.432 \times 10^{-12} \times 3333 \times \frac{12}{27.67 \times 10^{-6}} = 0.624 \text{ mV}$$

$$V_{ind,stall} = L_m \cdot \frac{I_{stall}}{\tau_{comm}} = 63.56 \times 10^{-9} \times \frac{0.65}{27.67 \times 10^{-6}} = 1.493 \text{ mV}$$

$$V_{noise,stall} = V_{cap,stall} + V_{ind,stall} = 0.624 + 1.493 = 2.117 \text{ mV}$$

5.3.2 Nominal (loaded) Case

In the nominal operation case (max current for thermal limit), each commutation event reverses the current direction in one winding, so the current transitions from $+I_N$ to $-I_N$, giving a total step of $\Delta I = 2I_N = 2 \times 0.225 = 0.45 \text{ A}$.

Unlike the stall case, V_{spike} no longer simplifies to V_N since $\Delta I \neq I_{stall}$. The spike on the motor lead is:

$$V_{spike,nom} = L_{motor} \cdot \frac{\Delta I}{\tau_{comm}} = \Delta I \cdot R_{motor} = 0.45 \times 18.2 = 8.19 \text{ V}$$

The capacitive and inductive crosstalk contributions are (with $R = 3333 \Omega$, in the worst case : I²C line with 3 devices):

$$V_{cap,nom} = C_m \cdot R_{pull} \cdot \frac{V_{spike,nom}}{\tau_{comm}} = 0.432 \times 10^{-12} \times 3333 \times \frac{8.19}{27.67 \times 10^{-6}} = 0.596 \text{ mV}$$

$$V_{ind,nom} = L_m \cdot \frac{\Delta I}{\tau_{comm}} = 63.56 \times 10^{-9} \times \frac{0.45}{27.67 \times 10^{-6}} = 1.034 \text{ mV}$$

$$V_{noise,nom} = V_{cap,nom} + V_{ind,nom} = 0.596 + 1.034 = 1.63 \text{ mV}$$

The crosstalk voltages computed above represent peak transient amplitudes occurring during individual commutation events. This is consistent with the NXP UM10204 [31] noise margin specification, which defines peak voltage bounds on the I²C bus lines : for fast I²C when the signal is pulled low

$$V_{noise,LOW} = V_{nL} = 0.1 \cdot V_{DD} = 330 \text{ mV}$$

and

$$V_{noise,HIGH} = V_{nH} = 0.2 \cdot V_{DD} = 660 \text{ mV}$$

, when the signal is pulled high.

We see that for this specific study case, the most constraining margin (stall, I²C LOW) is at least 150 times higher than the noise the system might experience. Against the HIGH margin, the safety factor exceeds 310.

This remains quite theoretical, limited by models of the physics and conservative approximations. We therefore tested this with an unloaded motor on the experimental setup by running the cables close by and checking for abnormal readings, which was not the case. We were however unable to test the nominal (loaded) or stall case, as the rest of the student association was preparing for its unveiling event and needed the motors.

5.4 Shock Detection Threshold

The goal of this safety component is to detect two critical events: the prosthetic detaching and falling, or the patient falling onto the outstretched prosthetic arm, in which cases the prosthetic would experience large accelerations, its integrity could be compromised, and it should be turned off.

As mentioned in the literature review (section 2.2), normal life activities can generate accelerations at the wrist up to 10 G's whereas events like a punch or falling on one's outstretched arm can generate accelerations of 80-230 G's.

To estimate the acceleration experienced specifically by the prosthetic arm falling freely, a first-principles calculation is used. Considering a free fall from $h = 1$ m and an assumed impact time of $t_{impact} = 5$ ms (hard material), the speed difference using a simple no drag model would be $\Delta v = \sqrt{2gh} = 4.43$ m/s and the corresponding peak acceleration using a half sine model is estimated at $a_{peak} = \frac{\pi \Delta v}{2t_{impact}} = 1391.7$ m/s² ≈ 142 g.

A threshold of 16 g is selected, coinciding with the maximum full-scale range of the LIS3DH accelerometer, which was itself chosen precisely because this range sits in the gap between normal daily activity levels and the minimum expected fall accelerations. Any event exceeding 16 g saturates the sensor and still reliably triggers the cutoff, regardless of the true peak magnitude. It was chosen so that it sits well above most normal daily activities, therefore mitigating false positives (which are highly undesirable as the arm will be shutdown), and well below the accelerations expected in the target shutdown events.

However, there are limitations. First, since this project is only a prototype, it remains unclear exactly where and how the sensor will be mounted on the prosthetic. Depending on that, some damping might be introduced, attenuating the signal at the sensor relative to the true surface acceleration. This will require controlled drop testing with the fully assembled prosthetic and the proposed safety sensing system (and possibly retuning the threshold).

Second, there is an ambiguous zone of high impact events, like hard knocks, which should not be categorized as safety compromising, but might trigger a shutdown. This is inevitable and considered acceptable, since an unexpected power cut during mishandling is less critical than a missed cut during a genuine fall. This may also be partially mitigated with full assembly testing and tuning.

5.5 Redundancy

As this project is a safety system, some amount of robustness was added by having two separate I²C lines and an extra 1-Wire line (3 more thermal sensors), with the sensors almost equally distributed among them. Some of the sources of failure could be a broken connector, a cut/-damaged cable, a solder joint breaking after mechanical stress cycle, a sensor malfunction that holds the line low, or physical damage causing a short that pulls the entire bus low.

This is impossible to quantify at this stage of the project, as it is a prototype and all of these failure potentials will look different on the real implementation. Furthermore, they are difficult to mathematically account for.

For all these reasons, the doubling of the I²C lines and extra 1-Wire line are a good first line of defense against such failures.

Together, the sensing rate analysis, bus integrity margins, noise margin, and line redundancy provide a quantitative basis for the engineering choices made in this safety monitoring system.

6 Results and Validation

The test bench successfully reads all the sensors, sending the data as expected to the GUI. The interrupts function correctly and the corresponding alert flags are indeed displayed on the GUI. The power cut flag is also displayed on the GUI as well as on the MCU using a red built-in LED.

The system is robust to the electromagnetic noise generated from an unloaded motor. Further testing would be required to fully assess the robustness of the proposed safety system. But much of that testing will require the safety system to be mounted on the arm and therefore redesigned.

The current voltage and power readings match closely the lab power source's readings. The temperature readings are consistent among themselves which is a good testament to their correct calibration and function. In longer experiments, using two temperature measurements on the motor running without load (other than the large reduction ratio, 203:1), the temperature rises and then settles, as expected.

Threshold validation was performed by artificially inducing fault states. The overcurrent interrupt successfully triggered at 40 mA (demonstration thresholds). The accelerometer reliably flagged a 4 g impact when subjected to a hard tap, while remaining inactive during normal handling. The thermal threshold triggered as the DS18B20 sensor crossed 28.0°C and humidity crossed 50%.

While demonstration thresholds were used for the testbench validation above, table 3 summarizes the final design thresholds intended for the fully integrated prosthetic. Figure 3, shows the final wiring diagram of the proposed safety system and figure 4 shows an image of the final prototype. Figure 5 shows a screenshot of the GUI with the temperature and humidity flags set.

Failure Mode	Sensor & Bus	Threshold	Response Path
Overcurrent / Short	INA260 (I ² C)	> 5.3 A (config.)	HW Interrupt (< 1.3 ms)
High Impact / Fall	LIS3DH (I ² C)	16 g	HW Interrupt (~ 5 ms)
Socket Overheating	DS18B20 (1-Wire)	41 °C	2 Hz Polling Loop
Battery Overheating	DS18B20 (1-Wire)	60 °C	2 Hz Polling Loop
Motor Overheating	DS18B20 (1-Wire)	85 °C (Casing)	2 Hz Polling Loop
Ambient / Electronics	SHT41 & Si7021 (I ² C)	65 °C	2 Hz Polling Loop

Table 3: System Safety Parameters Summary

7 Future work

This report and project is only the first step in the larger work of designing and integrating a safety system for the N-Pulse forearm prosthetic. This section covers some ideas for future work on this project.

The current prototype would benefit from thorough testing of the systems resilience to electromagnetic noise using several fully loaded or stalled motors. This was not possible as the rest of the team was using the motors during the end of the semester for the actual prosthetic prototype. On a similar note, the thermal model and assumptions of the motors should be validated under load, specifically the maximum temperature and how fast it gets there under stall and nominal loaded conditions.

Two planned components, the electronically controlled power switch and the MicroSD, were not delivered in time and remain for future implementation. Their integration would complete

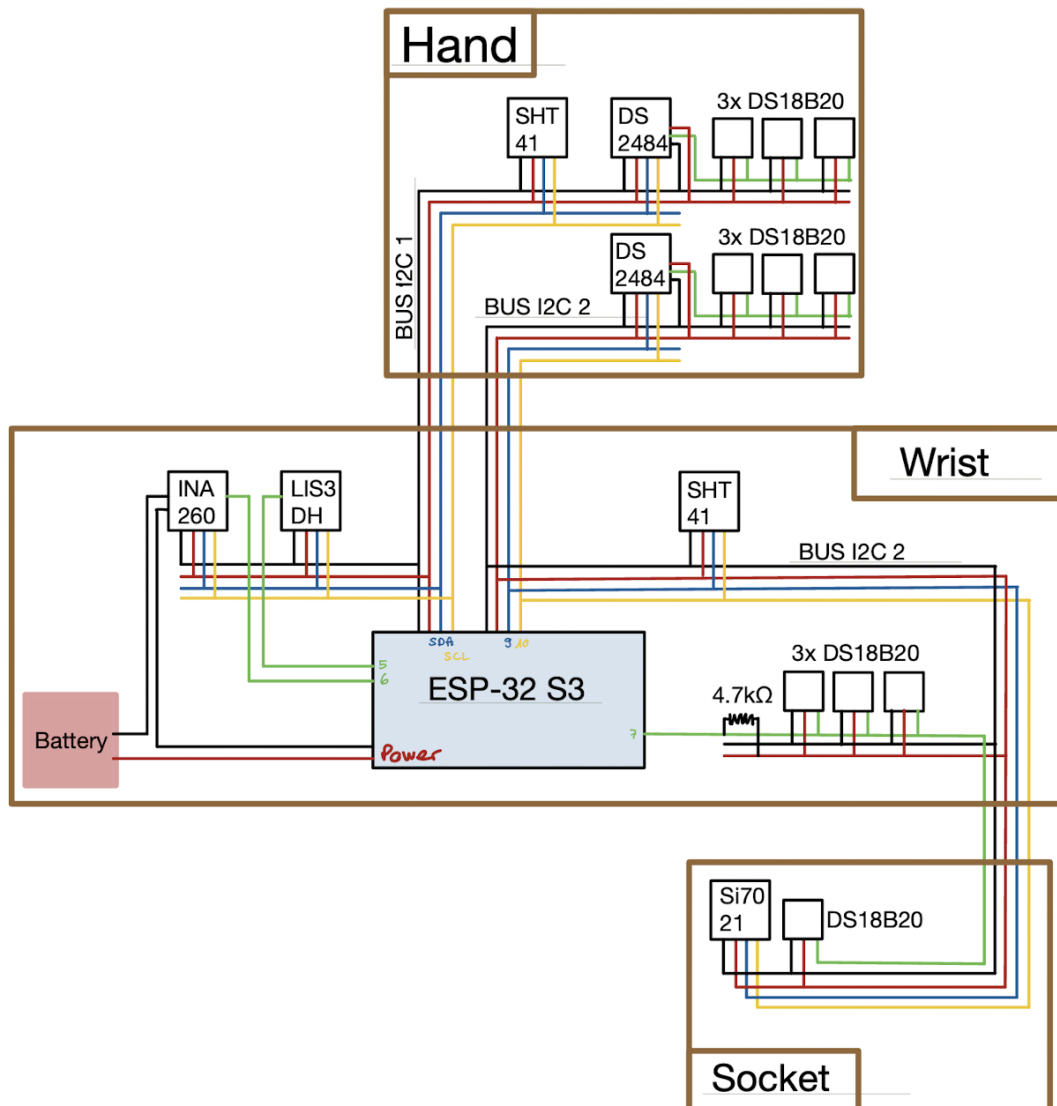


Figure 3: Test bench wiring schematic

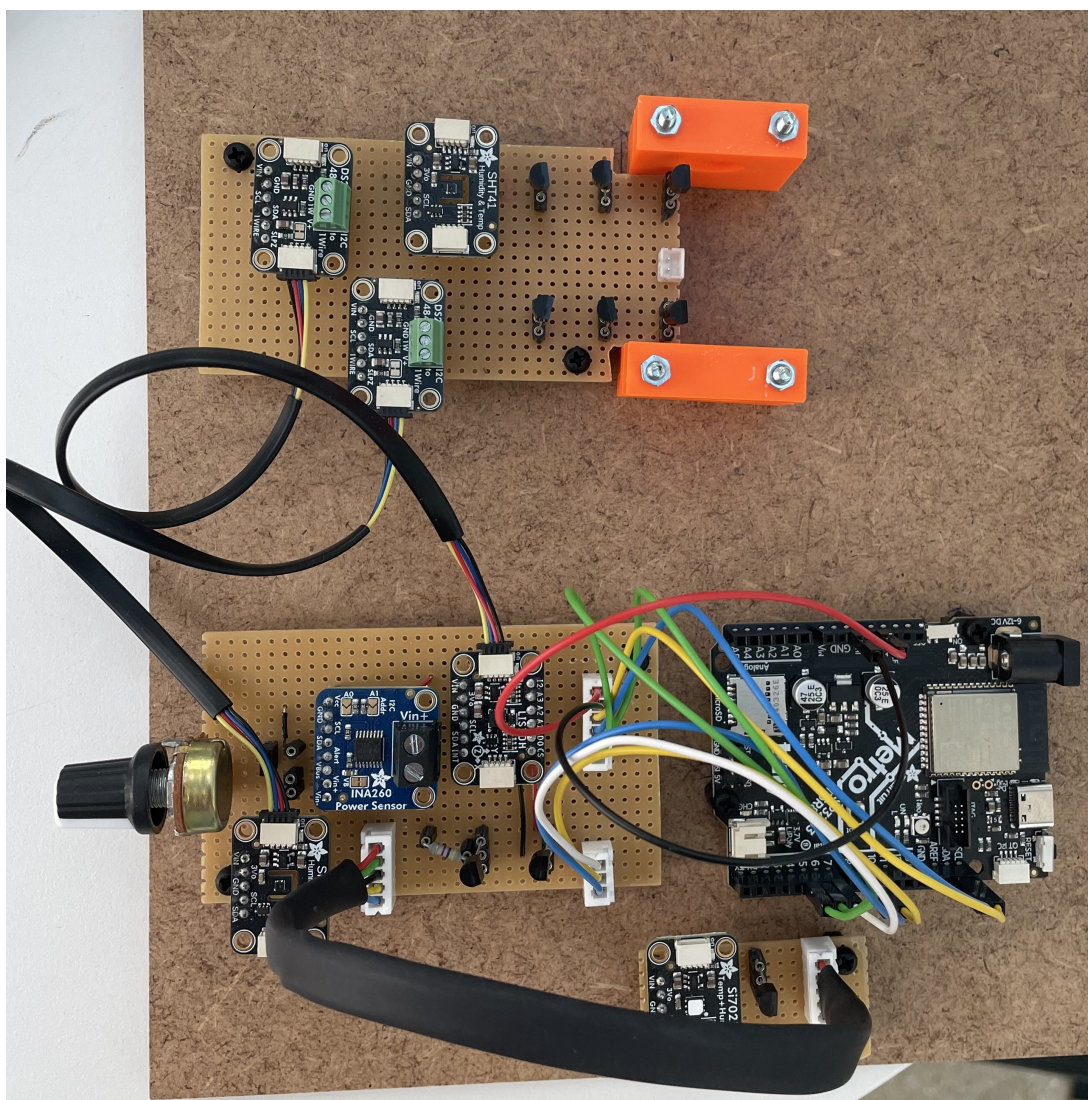


Figure 4: Top view image of the test bench

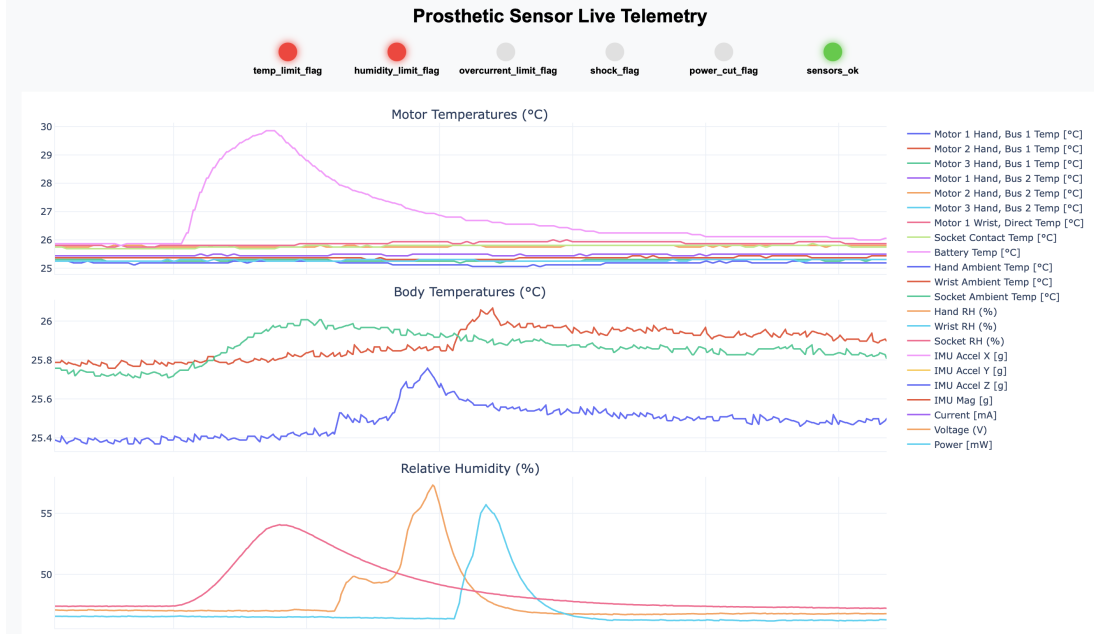


Figure 5: Screenshot of the graphical user interface with the temperature and humidity flags set

the autonomous fault response and data logging capabilities of the system.

For further development, this project would require a redesign for it to be implemented on the actual prosthetic. Designing custom PCB's would allow to use only the raw sensors and get rid of the rather large breakout boards. There could be one for the hand, communicating with the two 1-Wire buses, transferring them to I²C and containing the RH and temperature sensor (SHT41). The forearm PCB could contain the MCU, MicroSD card slot, current sensor, accelerometer and one more RH and temperature sensor (SHT41). Finally, the socket could also benefit from a miniaturized version of the Adafruit Si7021 and 1-Wire to I²C adapter for the socket contact temperature sensor. This would require close collaboration with the rest of the team to dial in placement and enough space for the components and boards.

8 Conclusion

This project covered the design and prototyping of an integrated safety monitoring system for a transradial prosthetic arm. Starting from an analysis of the most relevant failure risks: overcurrent, motor and battery overheating, ambient temperature, socket contact temperature, humidity, and high-impact events, some sensors were selected, justified, and assembled into a working prototype.

The hardware architecture distributes sensing across the hand, forearm, and socket, using two I²C buses and a 1-Wire bus for redundancy. Dedicated hardware interrupt lines handle the most time-critical faults (overcurrent and shock), with reaction times of approximately 1.3 ms and 5 ms respectively. Slower environmental metrics are monitored at 2 Hz, which was shown to be sufficient given the thermal time constants involved.

The prototype test bench successfully demonstrated all sensor readings, interrupt triggering, and fault flag reporting through a live visualization GUI. Electromagnetic noise from an unloaded motor was found to be well within the calculated safety margins.

The safety system is designed to be modular and expandable, compatible with future iterations of the prosthesis. Once integrated and tested on the full arm, it will help provide some of the safety guarantees necessary for user testing of the N-Pulse prosthetic arm.

References

- [1] R. Edwards, L. Murray, and A. Buis, “The role of human thermoregulation in thermal discomfort in lower-limb prosthetics: A scoping review,” *Canadian Prosthetics & Orthotics Journal*, vol. 8, no. 1, p. 43073, 2025.
- [2] R. J. Williams, *Exploring Thermal Discomfort Amongst Lower-Limb Prosthesis Wearers*. PhD thesis, University College London, 2020.
- [3] Limbs 4 life, Empowering Amputees, “Temperature control with a prosthetic limb.” <https://www.limbs4life.org.au/news/temperature-control-with-a-prosthetic-limb>.
- [4] Advanced Energy Industries, “Design considerations for maximum allowable temperature as per safety standards IEC 60601-1, IEC 60950-1 and IEC 61010-1.” https://www.advancedenergy.com/getmedia/8544158d-181a-4083-814e-90ffb75cf298/an_maximum_allowable_temperature.pdf.
- [5] B. J. Hafner and J. E. Sanders, “Considerations for development of sensing and monitoring tools to facilitate treatment and care of persons with lower-limb loss: a review,” *Journal of Rehabilitation Research and Development*, vol. 51, no. 1, pp. 1–14, 2014.
- [6] University of Bath, “Raw Acceleration from Wrist- and Hip-Worn Accelerometers Corresponds with Mechanical Loading in Children and Adolescents,” *PMC Open Access*, 2023.
- [7] A. Jovanovski and B. Stappenbelt, “Measuring the boxing punch: Development and calibration of a non-embedded in-glove piezo-resistive sensor,” *Proceedings*, vol. 49, no. 1, 2020.
- [8] J. Chiu and S. N. Robinovitch, “Prediction of upper extremity impact forces during falls on the outstretched hand,” *Journal of Biomechanics*, vol. 31, no. 12, pp. 1169–1176, 1998.
- [9] N. I. M. Amir, R. A. Dziyauddin, N. Mohamed, N. S. N. Ismail, H. M. Kaidi, N. Ahmad, and M. A. M. Izhar, “Fall detection system using wearable sensor devices and machine learning: A review,” *TechRxiv*, vol. 2024, no. 0319, 2024.
- [10] Oxeltech, “Design guide for battery safety in wearable electronics.” <https://oxeltech.de/design-guide-for-battery-safety-in-wearable-electronics/>, 2024.
- [11] D. Pont, M. A. Gonzalez, A. Contreras-González, and M. Ferre, “Predictor-based current limitation method for a dc motor-actuated upper-limb rehabilitation exoskeleton,” *IEEE Access*, vol. 10, pp. 1–1, 01 2022.
- [12] Adafruit Industries, “Adafruit metro ESP32-S3 guide.” <https://cdn-learn.adafruit.com/downloads/pdf/adafruit-metro-esp32-s3.pdf>.
- [13] Espressif Systems, “ESP32-S3-WROOM-1 and ESP32-S3-WROOM-1U datasheet.” https://documentation.espressif.com/esp32-s3-wroom-1_wroom-1u_datasheet_en.pdf.
- [14] Sensirion AG, “Datasheet SHT4x: Humidity and Temperature Sensor.” https://cdn-learn.adafruit.com/assets/assets/000/117/503/original/Datasheet_SHT4x.pdf, 2022.
- [15] K. Rembor, “Adafruit Sensirion SHT40, SHT41 & SHT45 Temperature & Humidity Sensors.” <https://cdn-learn.adafruit.com/downloads/pdf/adafruit-sht40-temperature-humidity-sensor.pdf>, 2025.
- [16] Silicon Laboratories, “Si7021 I²C humidity and temperature sensor datasheet.” <https://cdn.sparkfun.com/datasheets/Sensors/Weather/Si7021.pdf>.

- [17] Adafruit Industries, “Adafruit Si7021 temperature and humidity sensor breakout guide.” <https://cdn-learn.adafruit.com/downloads/pdf/adafruit-si7021-temperature-plus-humidity-sensor.pdf>.
- [18] Samsung SDI, “INR18650-35E lithium-ion cell datasheet.” <https://www.orbtronic.com/content/samsung-35e-datasheet-inr18650-35e.pdf>.
- [19] Elsmar Quality Forum, “IEC 60601-1 clause 11.1.2 — maximum temperature of applied parts.” <https://elsmar.com/elsmarqualityforum/threads/11-1-2-maximum-temperature-of-applied-parts.87215/>.
- [20] Faulhaber Group, “DC-Micromotors Series 1218... SXR Datasheet.” https://www.faulhaber.com/fileadmin/Import/Media/EN_1218_SXR_FCH.pdf.
- [21] Maxim Integrated, “DS18B20 Programmable Resolution 1-Wire Digital Thermometer.” <https://www.analog.com/media/en/technical-documentation/data-sheets/ds18b20.pdf>, 2019.
- [22] Maxim Integrated, “DS2484 Single-Channel I²C-to-1-Wire Bridge Device.” <https://www.analog.com/media/en/technical-documentation/data-sheets/ds2484.pdf>, 2009.
- [23] Adafruit Industries, “Adafruit DS2484 I²C-to-1-wire bus adapter breakout guide.” <https://cdn-learn.adafruit.com/downloads/pdf/adafruit-ds2484-i2c-to-1-wire-bus-adapter-breakout.pdf>.
- [24] Texas Instruments, “INA260 Precision Digital Current and Power Monitor with Low-Drift, Precision Integrated Shunt.” <https://www.ti.com/lit/ds/symlink/ina260.pdf>, 2023.
- [25] Adafruit Industries, “Adafruit INA260 current, voltage and power sensor breakout guide.” <https://cdn-learn.adafruit.com/downloads/pdf/adafruit-ina260-current-voltage-power-sensor-breakout.pdf>.
- [26] STMicroelectronics, “LIS3DH MEMS Digital Output Motion Sensor: Ultra-Low-Power High-Performance 3-Axis Nano Accelerometer.” <https://www.st.com/resource/en/datasheet/lis3dh.pdf>, 2016.
- [27] Adafruit Industries, “Adafruit LIS3DH triple-axis accelerometer breakout guide.” <https://cdn-learn.adafruit.com/downloads/pdf/adafruit-lis3dh-triple-axis-accelerometer-breakout.pdf>.
- [28] J. Kim, A. Mallarapu, and S. Santhanagopalan, “Transport processes in a li-ion cell during an internal short-circuit,” *Journal of The Electrochemical Society*, vol. 167, p. 090554, jun 2020.
- [29] ScienceDirect, “Overcurrent — an overview.” <https://www.sciencedirect.com/topics/engineering/overcurrent>. Accessed: 2026-06-05.
- [30] Texas Instruments, “Overcurrent protection enables more efficient and reliable systems,” tech. rep., Texas Instruments, 2016.
- [31] NXP Semiconductors, “UM10204: I²C-bus Specification and User misc, Rev. 7.” <https://www.nxp.com/docs/en/user-guide/UM10204.pdf>, Oct. 2021.
- [32] Schemalyzer, “I²C Pull-up Resistor Calculator.” <https://www.schemalyzer.com/en/tools/i2c-pullup-calculator>, 2025.

- [33] 3M Electronic Solutions Division, “3M Round Conductor Flat Cable, Series 3365, Technical Data Sheet TS-0080.” <https://multimedia.3m.com/mws/media/220520/3m-round-conductor-flat-cable-050in-3365-series-ts0080.pdf>.
- [34] Cornell University, ECE303, “Lecture 6: Capacitance.” <https://courses.cit.cornell.edu/ece303/Lectures/lecture6.pdf>.

Annex

Main Firmware

Main Sketch (ProsthesisSafety.ino)

```
1 #include "Config.h"
2 #include "DataTypes.h"
3 #include "Utils.h"
4 #include "Setup.h"
5 #include "Sensors.h"
6 #include "Safety.h"
7 #include "Logging.h"
8
9 void setup() {
10     Serial.begin(115200);
11     while (!Serial) delay(10); // wait for USB serial on ESP32-S3
12     delay(500);
13     Serial.println("[SETUP] Booting...");
14
15     // Output pins
16     pinMode(LED_BUILTIN, OUTPUT);
17     pinMode(Config::PIN_POWER_CUT, OUTPUT);
18     digitalWrite(Config::PIN_POWER_CUT, HIGH); // HIGH = power on (active-low
19         switch)
20
21     // Start both I2C buses
22     Wire.begin();
23     Wire.setClock(400000); // 400kHz fast mode
24     Wire1.begin(Config::SDA1_PIN, Config::SCL1_PIN);
25     Wire1.setClock(400000);
26
27     // Init each sensor group; AND results so any failure marks sensors_detected =
28     // false
29     sensors_detected &= setupHumiditySensors();
30     sensors_detected &= setupDS2484Line(line_bus0);
31     sensors_detected &= setupDS2484Line(line_bus1);
32     sensors_detected &= setupDirectOneWireLine();
33     sensors_detected &= setupINA260();
34     sensors_detected &= setupLIS3DH();
35     safety.sensors_ok = sensors_detected;
36
37     initSDLogging();
38
39     // Kick off the first DS18B20 conversion immediately so valid readings
40     // are ready after ~94ms instead of waiting for the second log cycle
41     startDS2484Conversion(line_bus0);
42     startDS2484Conversion(line_bus1);
43     startDirectOneWireConversion();
44
45     if (Config::PRINT_CSV_HEADER_AT_BOOT) printCSVHeader();
46
47     lastLogMs = millis();
48 }
49
50 void loop() {
51     // 1. Handle any interrupt flags raised since last loop (fast, non-blocking)
52     handleCurrentInterrupt();
53     handleIMUInterrupt();
54
55     // 2. Log at fixed period (100ms default)
56     if (millis() - lastLogMs >= Config::LOG_PERIOD_MS) {
```

```

55     lastLogMs += Config::LOG_PERIOD_MS; // increment rather than = millis() to
        avoid drift
56     logOncePerPeriod();
57 }
58
59 // 3. Reflect alarm state on the built-in LED
60 updateStatusLED();
61 }

```

Configuration (Config.h)

```

1  #pragma once
2
3  // All tunable parameters live here. Edit this file to change behavior.
4  namespace Config {
5
6      // ---- Physical pin assignments ----
7      static const uint8_t SDA1_PIN           = 9;    // I2C bus 1 data
8      static const uint8_t SCL1_PIN           = 10;   // I2C bus 1 clock
9      static const uint8_t DIRECT_ONEWIRE_PIN = 7;    // 1-Wire bus (no bridge
        chip)
10     static const uint8_t PIN_INT_IMU         = 5;    // LIS3DH interrupt -> MCU
11     static const uint8_t PIN_INT_CURRENT     = 6;    // INA260 alert -> MCU
12     static const uint8_t PIN_POWER_CUT       = 11;   // pull LOW to cut load
        power
13     static const uint8_t PIN_SD_CS           = 12;   // SD chip-select (future)
14
15     // ---- I2C addresses ----
16     static const uint8_t ADDR_DS2484         = 0x18;
17     static const uint8_t ADDR_LIS3DH         = 0x19;
18     static const uint8_t ADDR_INA260         = 0x40;
19     // SHT41 uses its default address, no override needed
20
21     // ---- How many DS18B20s to scan per 1-Wire line ----
22     static const uint8_t MAX_SENSORS_PER_LINE = 3;
23
24     // ---- Timing ----
25     static const uint32_t LOG_PERIOD_MS       = 500; // how often the
        main loop logs
26     static const uint32_t DIRECT_DS18B20_CONVERSION_MS = 94; // DS18B20 needs
        94ms (9-bit) or 750ms (12-bit) to convert
27     static const uint32_t DS2484_DS18B20_CONVERSION_MS = 94; // same for bridge
        -connected ones
28
29     // ---- Safety limits trigger alerts above these values ----
30     static constexpr float MAX_TEMP_C         = 85.0f; // C (normal use:
        85.0)
31     static constexpr float MAX_RH_PERCENT      = 50.0f; // %RH (normal use:
        95.0)
32     static constexpr float MAX_CURRENT_MA      = 40.0f; // mA (normal use:
        4000.0)
33
34     // ---- What to do when a limit is crossed ----
35     // set to true to actually cut power on that event
36     static const bool ENABLE_POWER_CUT_ON_OVERCURRENT = false;
37     static const bool ENABLE_POWER_CUT_ON_OVERHEAT    = false;
38     static const bool ENABLE_POWER_CUT_ON_OVERHUMIDITY = false;
39     static const bool ENABLE_POWER_CUT_ON_CONFIRMED_SHOCK = false;
40
41     // ---- Impact detection tuning (LIS3DH) ----
42     static constexpr float IMPACT_THRESHOLD_G        = 4.0f; // g to trigger
        interrupt

```



```

43 // static constexpr float CONFIRM_PEAK_G          = 5.5f; // g to confirm as
    real shock
44 static const uint8_t IMPACT_DURATION_SAMPLES = 1; // consecutive
    samples above threshold
45 static const uint8_t ODR_BITS_200HZ            = 0x60; // output data rate
    register value
46 static const uint8_t RANGE_BITS_16G           = 0x30; // 16g range
    register value
47
48 // ---- Sensor mapping: hard-coded 1-Wire addresses and per-sensor thresholds
    ----
49 // Run once with thresholds zeroed to discover your sensors, then copy their
    addresses here
50 enum class SensorPurpose { BATTERY, SOCKET, MOTOR, UNKNOWN };
51
52 struct SensorMapping {
53     uint8_t address[8];
54     SensorPurpose purpose;
55     float temp_limit_c;
56 };
57
58 // Bus 0 (DS2484) all motor sensors
59 static constexpr SensorMapping bus0_sensors[] = {
60     {{ 0x28, 0xFF, 0x64, 0x0E, 0x6C, 0x7E, 0xBA, 0x6B }, SensorPurpose::MOTOR,
        85.0f },
61     {{ 0x28, 0xFF, 0x64, 0x0E, 0x6D, 0x93, 0xDC, 0x84 }, SensorPurpose::MOTOR,
        85.0f },
62     {{ 0x28, 0xFF, 0x64, 0x0E, 0x7B, 0x21, 0xAF, 0x96 }, SensorPurpose::MOTOR,
        85.0f },
63 };
64 static constexpr uint8_t bus0_sensors_count = sizeof(bus0_sensors) / sizeof(
    bus0_sensors[0]);
65
66 // Bus 1 (DS2484) all motor sensors
67 static constexpr SensorMapping bus1_sensors[] = {
68     {{ 0x28, 0xFF, 0x64, 0x0E, 0x6D, 0x49, 0x39, 0xED }, SensorPurpose::MOTOR,
        85.0f },
69     {{ 0x28, 0xFF, 0x64, 0x0E, 0x6D, 0xC5, 0x78, 0x97 }, SensorPurpose::MOTOR,
        85.0f },
70     {{ 0x28, 0xFF, 0x64, 0x0E, 0x6D, 0xE5, 0xC2, 0x1A }, SensorPurpose::MOTOR,
        85.0f },
71 };
72 static constexpr uint8_t bus1_sensors_count = sizeof(bus1_sensors) / sizeof(
    bus1_sensors[0]);
73
74 // Direct line (no DS2484 bridge) battery, socket, motor
75 static constexpr SensorMapping direct_sensors[] = {
76     {{ 0x28, 0xFF, 0x64, 0x0E, 0x6D, 0x54, 0x36, 0xC9 }, SensorPurpose::BATTERY,
        45.0f },
77     {{ 0x28, 0xFF, 0x64, 0x0E, 0x6D, 0x86, 0x2C, 0xE3 }, SensorPurpose::SOCKET,
        41.0f }, // 41
78     {{ 0x28, 0xFF, 0x64, 0x0E, 0x6D, 0x5B, 0xB0, 0x80 }, SensorPurpose::MOTOR,
        85.0f },
79 };
80 static constexpr uint8_t direct_sensors_count = sizeof(direct_sensors) /
    sizeof(direct_sensors[0]);
81
82 // ---- Serial / logging ----
83 static const bool PRINT_CSV_HEADER_AT_BOOT        = false;
84 static const uint16_t PRINT_CSV_HEADER_EVERY_N_LOGS = false; // reprint
    header every N lines
85 static const bool ENABLE_SD_LOGGING                = false; // SD not wired
    yet

```

Data Types (DataTypes.h)

```

1  #pragma once
2
3  // Standard libs
4  #include <Wire.h>
5  #include <SPI.h>
6  #include <math.h>    // isnan(), sqrtf()
7  #include <string.h>  // memcpy()
8
9  // Sensor libraries
10 #include <Adafruit_Sensor.h>
11 #include <7semi_SHT4x.h>
12 #include "Adafruit_INA260.h"
13 #include "Adafruit_Si7021.h"
14 #include <Adafruit_LIS3DH.h>
15 #include "Adafruit_DS248x.h"
16 #include <OneWire.h>
17 #include <DallasTemperature.h>
18
19 #include "Config.h"
20
21 // DS18B20 1-Wire command bytes (from datasheet)
22 #define DS18B20_FAMILY_CODE      0x28  // first ROM byte identifies the
    sensor model
23 #define DS18B20_CMD_CONVERT_T    0x44  // tell sensor to start measuring
24 #define DS18B20_CMD_MATCH_ROM    0x55  // address one specific sensor by its
    ROM
25 #define DS18B20_CMD_READ_SCRATCHPAD 0xBE  // read the result back
26
27 // ---- Data structs ----
28 // Each sensor type has its own reading struct.
29 // ok = false means the read failed; values will be NAN.
30
31 struct TH_Reading {
32     bool ok      = false;
33     float temp_c  = NAN;
34     float rh_percent = NAN;
35 };
36
37 struct PowerReading {
38     bool ok      = false;
39     float current_ma = NAN;
40     float bus_mv    = NAN;
41     float power_mw  = NAN;
42 };
43
44 struct IMUReading {
45     bool ok      = false;
46     float ax_g   = NAN; // x-axis acceleration in g
47     float ay_g   = NAN;
48     float az_g   = NAN;
49     float mag_g  = NAN; // vector magnitude: sqrt(ax^2+ay^2+az^2)
50 };
51
52 struct SafetyFlags {
53     bool any_temp_limit      = false; // latches once triggered
54     bool any_humidity_limit  = false; // latches once triggered
55     bool overcurrent_limit   = false; // latches once triggered
56     bool shock_flag          = false; // latches once triggered

```

```

57     bool overcurrent_irq_latched = false; // set when INA260 fires its alert pin
58     bool power_cut_flag          = false; // set once power has been cut
59     bool sensors_ok              = true;
60 };
61
62 // One entry per DS2484-bridged 1-Wire line
63 struct DS2484Line {
64     const char*      name;           // label for debug prints
65     TwoWire*         wire;           // which I2C bus the DS2484 sits on
66     Adafruit_DS248x* bridge;         // the DS2484 driver object
67     uint8_t          roms[Config::MAX_SENSORS_PER_LINE][8]; // 64-bit ROM of each
        found sensor
68     uint8_t          found           = 0;
69     float             temperatures[Config::MAX_SENSORS_PER_LINE] = {NAN, NAN, NAN};
70     bool              present        = false;
71     bool              conversionPending = false; // true while waiting for
        temperature conversion
72     unsigned long     conversionStartMs = 0;
73 };
74
75 // The one line wired directly (no DS2484 bridge)
76 struct DirectOneWireLine {
77     DeviceAddress addresses[Config::MAX_SENSORS_PER_LINE];
78     uint8_t          found           = 0;
79     float             temperatures[Config::MAX_SENSORS_PER_LINE] = {NAN, NAN, NAN};
80     bool              present        = false;
81     bool              conversionPending = false;
82     unsigned long     conversionStartMs = 0;
83 };
84
85 // ---- Sensor driver objects ----
86 // Defined once in DataTypes.cpp; extern here so all files can use them.
87 extern SHT4x_7semi      sht41_bus0;
88 extern SHT4x_7semi      sht41_bus1;
89 extern Adafruit_Si7021  si7021_bus1;
90 extern Adafruit_INA260  ina260;
91 extern Adafruit_LIS3DH  imu;
92 extern Adafruit_DS248x  ds2484_bus0;
93 extern Adafruit_DS248x  ds2484_bus1;
94 extern OneWire           oneWireDirect;
95 extern DallasTemperature directTemps;
96
97 // ---- 1-Wire line instances ----
98 extern DS2484Line        line_bus0;
99 extern DS2484Line        line_bus1;
100 extern DirectOneWireLine line_direct;
101
102 // ---- Latest readings (updated each log cycle) ----
103 extern TH_Reading        reading_sht41_bus0;
104 extern TH_Reading        reading_sht41_bus1;
105 extern TH_Reading        reading_si7021_bus1;
106 extern PowerReading      reading_power;
107 extern IMUReading        reading_imu;
108 extern SafetyFlags       safety;
109
110 // ---- Interrupt flags ----
111 // Set to true inside the ISR, cleared and handled in the main loop.
112 // volatile = don't let the compiler cache these in a register
113 extern volatile bool overcurrent_irq;
114 extern volatile bool impact_irq;
115
116 // ---- Misc globals ----
117 extern bool             sensors_detected; // false if any sensor failed init

```

```

118 extern unsigned long lastLogMs;           // timestamp of last log line
119 extern uint32_t      logCounter;          // total lines logged so far

```

Data Types Implementation (DataTypes.cpp)

```

1  #include "DataTypes.h"
2
3  // Actual memory allocation for everything declared extern in DataTypes.h.
4  // Only this file defines them; all other files just reference them.
5
6  // ---- Sensor drivers ----
7  SHT4x_7semi      sht41_bus0;
8  SHT4x_7semi      sht41_bus1;
9  Adafruit_Si7021  si7021_bus1(&Wire1); // explicitly on bus 1
10 Adafruit_INA260   ina260;
11 Adafruit_LIS3DH   imu = Adafruit_LIS3DH();
12 Adafruit_DS248x   ds2484_bus0;
13 Adafruit_DS248x   ds2484_bus1;
14 OneWire            oneWireDirect(Config::DIRECT_ONEWIRE_PIN);
15 DallasTemperature directTemps(&oneWireDirect); // wraps OneWire for DS18B20
    convenience
16
17 // ---- 1-Wire lines ----
18 DS2484Line        line_bus0 = {"line_bus0", &Wire, &ds2484_bus0};
19 DS2484Line        line_bus1 = {"line_bus1", &Wire1, &ds2484_bus1};
20 DirectOneWireLine line_direct; // addresses filled during setup
21
22 // ---- Readings (start invalid; filled after first sensor poll) ----
23 TH_Reading        reading_sht41_bus0;
24 TH_Reading        reading_sht41_bus1;
25 TH_Reading        reading_si7021_bus1;
26 PowerReading      reading_power;
27 IMUReading        reading_imu;
28 SafetyFlags       safety;
29
30 // ---- Interrupt flags ----
31 volatile bool overcurrent_irq = false;
32 volatile bool impact_irq      = false;
33
34 // ---- Misc ----
35 bool          sensors_detected = true;
36 unsigned long lastLogMs        = 0;
37 uint32_t      logCounter       = 0;

```

Logging (Logging.h)

```

1  #pragma once
2  #include "DataTypes.h"
3
4  void printCSVHeader(); // column names row
5  void printCSVLine();   // one data row with current readings + flags
6  void logToSDCard();    // stub mirrors printCSVLine() to SD when enabled
7  void logOncePerPeriod(); // called by loop(); reads sensors, evaluates safety,
    then logs
8  void updateStatusLED(); // blink LED_BUILTIN if any alarm is active

```

Logging Implementation (Logging.cpp)

```

1  #include "Logging.h"
2  #include "Utils.h"
3  #include "Sensors.h"
4  #include "Safety.h"
5
6  void printCSVHeader() {
7      Serial.println(
8          "t_ms,"
9          "hand_temp_c,hand_rh,"
10         "wrist_temp_c,wrist_rh,"
11         "socket_temp_c,socket_rh,"
12         "bus0_ds1_c,bus0_ds2_c,bus0_ds3_c,"
13         "bus1_ds1_c,bus1_ds2_c,bus1_ds3_c,"
14         "battery,socket,direct_ds1_c,"
15         "imu_ax_g,imu_ay_g,imu_az_g,imu_mag_g,"
16         "current_ma,bus_mv,power_mw,"
17         "temp_limit_flag,humidity_limit_flag,overcurrent_limit_flag,"
18         "shock_flag,power_cut_flag,sensors_ok"
19     );
20 }
21
22 void printCSVLine() {
23     Serial.print(millis()); Serial.print(',');
24
25     // I2C humidity sensors
26     printFloatCSV(reading_sht41_bus0.temp_c);    Serial.print(',');
27     printFloatCSV(reading_sht41_bus0.rh_percent); Serial.print(',');
28     printFloatCSV(reading_sht41_bus1.temp_c);    Serial.print(',');
29     printFloatCSV(reading_sht41_bus1.rh_percent); Serial.print(',');
30     printFloatCSV(reading_si7021_bus1.temp_c);    Serial.print(',');
31     printFloatCSV(reading_si7021_bus1.rh_percent); Serial.print(',');
32
33     // DS18B20 temperatures    always print MAX_SENSORS_PER_LINE columns,
34     // filling with "nan" for slots where no sensor was found
35     for (uint8_t i = 0; i < Config::MAX_SENSORS_PER_LINE; i++) {
36         printFloatCSV(i < line_bus0.found ? line_bus0.temperatures[i] : NAN);
37         Serial.print(',');
38     }
39     for (uint8_t i = 0; i < Config::MAX_SENSORS_PER_LINE; i++) {
40         printFloatCSV(i < line_bus1.found ? line_bus1.temperatures[i] : NAN);
41         Serial.print(',');
42     }
43     for (uint8_t i = 0; i < Config::MAX_SENSORS_PER_LINE; i++) {
44         printFloatCSV(i < line_direct.found ? line_direct.temperatures[i] : NAN);
45         if (i < Config::MAX_SENSORS_PER_LINE - 1) Serial.print(',');
46     }
47
48     // IMU
49     Serial.print(','); printFloatCSV(reading_imu.ax_g);
50     Serial.print(','); printFloatCSV(reading_imu.ay_g);
51     Serial.print(','); printFloatCSV(reading_imu.az_g);
52     Serial.print(','); printFloatCSV(reading_imu.mag_g);
53
54     // Power
55     Serial.print(','); printFloatCSV(reading_power.current_ma);
56     Serial.print(','); printFloatCSV(reading_power.bus_mv);
57     Serial.print(','); printFloatCSV(reading_power.power_mw);
58
59     // Safety flags as 0/1
60     Serial.print(','); Serial.print(safety.any_temp_limit      ? 1 : 0);
61     Serial.print(','); Serial.print(safety.any_humidity_limit   ? 1 : 0);
62     Serial.print(','); Serial.print(safety.overcurrent_limit    ? 1 : 0);
63     Serial.print(','); Serial.print(safety.shock_flag           ? 1 : 0);

```

```

64     Serial.print(','); Serial.print(safety.power_cut_flag      ? 1 : 0);
65     Serial.print(','); Serial.println(safety.sensors_ok      ? 1 : 0);
66 }
67
68 void logToSDCard() {
69     if (!Config::ENABLE_SD_LOGGING) return;
70     // TODO: mirror printCSVLine() to an open SD File handle; flush every N lines
71     // to reduce wear
72 }
73
74 // One complete log cycle: read -> evaluate -> print
75 void logOncePerPeriod() {
76     // Read results from PREVIOUS conversion (already done)
77     finishDS2484Conversion(line_bus0);
78     finishDS2484Conversion(line_bus1);
79     finishDirectOneWireConversion();
80
81     readFastSensors();
82     evaluateSafetyConditions();
83     printCSVLine();
84     logToSDCard();
85
86     // Start NEXT conversion immediately after reading
87     startDS2484Conversion(line_bus0);
88     startDS2484Conversion(line_bus1);
89     startDirectOneWireConversion();
90
91     logCounter++;
92 }
93
94 // LED on = at least one alarm is active
95 void updateStatusLED() {
96     bool any_alarm =
97         safety.any_temp_limit      ||
98         safety.any_humidity_limit  ||
99         safety.overcurrent_limit   ||
100         safety.shock_flag          ||
101         safety.overcurrent_irq_latched ||
102         safety.power_cut_flag      ||
103         !safety.sensors_ok;
104
105     digitalWrite(LED_BUILTIN, any_alarm ? HIGH : LOW);
106 }
107
108 // One complete log cycle: read -> evaluate -> print
109 // void logOncePerPeriod() {
110 //     readFastSensors();
111 //     evaluateSafetyConditions();
112 //     // check if not none
113 //     if (Config::PRINT_CSV_HEADER_EVERY_N_LOGS) {
114 //         if (logCounter % Config::PRINT_CSV_HEADER_EVERY_N_LOGS == 0) {
115 //             printCSVHeader();
116 //         }
117 //     }
118 //     printCSVLine();
119 //     logToSDCard();
120 //     logCounter++;
121 // }
122
123 // // LED on = at least one alarm is active
124 // void updateStatusLED() {
125 //     bool any_alarm =
126 //         safety.any_temp_limit      ||

```

```

126 //      safety.any_humidity_limit      //
127 //      safety.overcurrent_limit      //
128 //      safety.shock_flag              //
129 //      safety.overcurrent_irq_latched//
130 //      safety.power_cut_flag          //
131 //      !safety.sensors_ok;
132
133 //      digitalWrite(LED_BUILTIN, any_alarm ? HIGH : LOW);
134 // }

```

Safety (Safety.h)

```

1 #pragma once
2 #include "DataTypes.h"
3 #include "Sensors.h"
4
5 // Check all current readings against limits; set safety flags accordingly
6 void evaluateSafetyConditions();
7
8 // Handle interrupt flags set by ISRs (call at top of loop())
9 void handleCurrentInterrupt();
10 void handleIMUInterrupt();

```

Safety Implementation (Safety.cpp)

```

1 #include "Safety.h"
2 #include "Utils.h"
3
4 void evaluateSafetyConditions() {
5     // Don't reset latching flags      they stay up once triggered
6     bool new_temp_limit      = false;
7     bool new_humidity_limit  = false;
8     bool new_overcurrent     = false;
9
10    // Mark sensor health      any single failure marks the system degraded
11    if (!reading_sht41_bus0.ok || !reading_sht41_bus1.ok ||
12        !reading_si7021_bus1.ok || !reading_power.ok || !reading_imu.ok) {
13        safety.sensors_ok = false;
14    }
15
16    // T/RH limit checks on I2C humidity sensors
17    if (aboveTempLimit(reading_sht41_bus0.temp_c) ||
18        aboveTempLimit(reading_sht41_bus1.temp_c) ||
19        aboveTempLimit(reading_si7021_bus1.temp_c)) {
20        new_temp_limit = true;
21    }
22
23    if (aboveHumidityLimit(reading_sht41_bus0.rh_percent) ||
24        aboveHumidityLimit(reading_sht41_bus1.rh_percent) ||
25        aboveHumidityLimit(reading_si7021_bus1.rh_percent)) {
26        new_humidity_limit = true;
27    }
28
29    // Temperature limit checks on DS18B20s across all 3 lines (with per-sensor
30    // limits)
31    for (uint8_t i = 0; i < line_bus0.found; i++) {
32        float limit = Config::MAX_TEMP_C;
33        getSensorTempLimit(line_bus0.roms[i], limit);
34        if (!isnan(line_bus0.temperatures[i]) && line_bus0.temperatures[i] > limit)
35            new_temp_limit = true;
36    }
37 }

```

```

35     }
36     for (uint8_t i = 0; i < line_bus1.found; i++) {
37         float limit = Config::MAX_TEMP_C;
38         getSensorTempLimit(line_bus1.roms[i], limit);
39         if (!isnan(line_bus1.temperatures[i]) && line_bus1.temperatures[i] > limit)
40             new_temp_limit = true;
41     }
42     for (uint8_t i = 0; i < line_direct.found; i++) {
43         float limit = Config::MAX_TEMP_C;
44         getSensorTempLimit((const uint8_t*)line_direct.addresses[i], limit);
45         if (!isnan(line_direct.temperatures[i]) && line_direct.temperatures[i] >
46             limit)
47             new_temp_limit = true;
48     }
49     // Software overcurrent check (INA260 polled reading, separate from the IRQ
50     path)
51     if (!isnan(reading_power.current_ma) && reading_power.current_ma > Config::
52         MAX_CURRENT_MA)
53         new_overcurrent = true;
54
55     // Latch temp and humidity flags (stay true once triggered)
56     safety.any_temp_limit = safety.any_temp_limit || new_temp_limit;
57     safety.any_humidity_limit = safety.any_humidity_limit || new_humidity_limit;
58     safety.overcurrent_limit = safety.overcurrent_limit || new_overcurrent;
59
60     // Trigger power cuts
61     if (new_temp_limit && Config::ENABLE_POWER_CUT_ON_OVERHEAT)
62         requestPowerCut("temperature_limit_exceeded");
63     if (new_humidity_limit && Config::ENABLE_POWER_CUT_ON_OVERHUMIDITY)
64         requestPowerCut("humidity_limit_exceeded");
65     if (new_overcurrent && Config::ENABLE_POWER_CUT_ON_OVERCURRENT)
66         requestPowerCut("overcurrent_(polled)");
67 }
68
69 void handleCurrentInterrupt() {
70     if (!overcurrent_irq) return;
71
72     noInterrupts(); overcurrent_irq = false; interrupts();
73
74     // Power cut already triggered from ISR; verify via I2C and log
75     if (ina260.alertFunctionFlag()) {
76         safety.overcurrent_irq_latched = true;
77     }
78 }
79
80 void handleIMUInterrupt() {
81     if (!impact_irq) return;
82
83     noInterrupts(); impact_irq = false; interrupts();
84
85     // Power cut already triggered from ISR; verify via I2C and log
86     uint8_t src = readIMUReg(0x31);
87     bool ia = src & 0x40; // bit 6 = IA (Interrupt Active): at least one
88     condition was met
89
90     if (ia) {
91         safety.shock_flag = true;
92     }
93 }

```


Sensors (Sensors.h)

```
1 #pragma once
2 #include "DataTypes.h"
3
4 // Single-shot reads for fast sensors (I2C, call each log cycle)
5 TH_Reading readSHT41Sensor(SHT4x_7semi& sensor);
6 TH_Reading readSi7021Sensor(Adafruit_Si7021& sensor);
7 PowerReading readPowerSensor();
8 IMUReading readIMUSensor();
9 void readFastSensors(); // calls all four above and stores results
10
11 // Non-blocking DS18B20 state machine (tied to log cycle).
12 // DS18B20 needs ~94ms to convert: start on log N, read on log N+1.
13 void startDS2484Conversion(DS2484Line& line);
14 void finishDS2484Conversion(DS2484Line& line);
15 void startDirectOneWireConversion();
16 void finishDirectOneWireConversion();
```

Sensors Implementation (Sensors.cpp)

```
1 #include "Sensors.h"
2
3 TH_Reading readSHT41Sensor(SHT4x_7semi& sensor) {
4     TH_Reading out;
5     // readTemperatureHumidity returns false on CRC error or timeout
6     out.ok = sensor.readTemperatureHumidity(out.temp_c, out.rh_percent);
7     return out;
8 }
9
10 TH_Reading readSi7021Sensor(Adafruit_Si7021& sensor) {
11     TH_Reading out;
12     out.rh_percent = sensor.readHumidity();
13     out.temp_c = sensor.readTemperature();
14     out.ok = !isnan(out.rh_percent) && !isnan(out.temp_c);
15     return out;
16 }
17
18 PowerReading readPowerSensor() {
19     PowerReading out;
20     out.current_ma = ina260.readCurrent();
21     out.bus_mv = ina260.readBusVoltage();
22     out.power_mw = ina260.readPower();
23     out.ok = !isnan(out.current_ma);
24     return out;
25 }
26
27 IMUReading readIMUSensor() {
28     IMUReading out;
29     sensors_event_t event;
30     imu.getEvent(&event); // fills event.acceleration in m/s
31     // Convert m/s to g (1g = 9.80665 m/s )
32     out.ax_g = event.acceleration.x / 9.80665f;
33     out.ay_g = event.acceleration.y / 9.80665f;
34     out.az_g = event.acceleration.z / 9.80665f;
35     out.mag_g = sqrtf(out.ax_g*out.ax_g + out.ay_g*out.ay_g + out.az_g*out.az_g);
36     out.ok = true;
37     return out;
38 }
39
40 // Read all fast sensors and store into global reading_* variables
41 void readFastSensors() {
```

```

42   reading_sht41_bus0  = readSHT41Sensor(sht41_bus0);
43   reading_sht41_bus1  = readSHT41Sensor(sht41_bus1);
44   reading_si7021_bus1 = readSi7021Sensor(si7021_bus1);
45   reading_power        = readPowerSensor();
46   reading_imu          = readIMUSensor();
47 }
48
49 // Phase 1: broadcast "start converting" to all sensors on this line, then set a
      timer
50 void startDS2484Conversion(DS2484Line& line) {
51     if (!line.present || line.conversionPending || line.found == 0) return;
52     for (uint8_t i = 0; i < line.found; i++) {
53         line.bridge->OneWireReset();
54         line.bridge->OneWireWriteByte(DS18B20_CMD_MATCH_ROM);
55         for (uint8_t j = 0; j < 8; j++) line.bridge->OneWireWriteByte(line.roms[i][j]
            );
56         line.bridge->OneWireWriteByte(DS18B20_CMD_CONVERT_T);
57     }
58     line.conversionPending = true;
59     line.conversionStartMs = millis();
60 }
61
62 // Phase 2: once 94ms have passed, read the scratchpad from each sensor
63 void finishDS2484Conversion(DS2484Line& line) {
64     if (!line.conversionPending) return;
65     if (millis() - line.conversionStartMs < Config::DS2484_DS18B20_CONVERSION_MS)
        return;
66
67     for (uint8_t i = 0; i < line.found; i++) {
68         line.bridge->OneWireReset();
69         line.bridge->OneWireWriteByte(DS18B20_CMD_MATCH_ROM);
70         for (uint8_t j = 0; j < 8; j++) line.bridge->OneWireWriteByte(line.roms[i][j]
            );
71         line.bridge->OneWireWriteByte(DS18B20_CMD_READ_SCRATCHPAD);
72
73         uint8_t data[9];
74         for (uint8_t k = 0; k < 9; k++) line.bridge->OneWireReadByte(&data[k]);
75         // Raw value is a signed 16-bit int: LSB in data[0], MSB in data[1]
76         // Divide by 16 because the last 4 bits are the fractional part ( $2^{-4} = 0.0625$  C /LSB)
77         int16_t raw = (data[1] << 8) | data[0];
78         line.temperatures[i] = (float)raw / 16.0f;
79     }
80     line.conversionPending = false;
81 }
82
83 // Same state machine for the direct (no bridge) line      uses DallasTemperature
      library instead
84 void startDirectOneWireConversion() {
85     if (!line_direct.present || line_direct.conversionPending || line_direct.found
        == 0) return;
86     directTemps.requestTemperatures(); // sends CONVERT_T to all devices on the
        bus
87     line_direct.conversionPending = true;
88     line_direct.conversionStartMs = millis();
89 }
90
91 void finishDirectOneWireConversion() {
92     if (!line_direct.conversionPending) return;
93     if (millis() - line_direct.conversionStartMs < Config::
        DIRECT_DS18B20_CONVERSION_MS) return;
94
95     for (uint8_t i = 0; i < line_direct.found; i++) {

```

```

96     line_direct.temperatures[i] = directTemps.getTempC(line_direct.addresses[i])
97     ;
98 }
99 line_direct.conversionPending = false;

```

Setup (Setup.h)

```

1  #pragma once
2  #include "DataTypes.h"
3  #include "Utils.h"
4
5  // One function per sensor group; each returns false if init fails
6  bool setupHumiditySensors();
7  bool setupDS2484Line(DS2484Line& line);
8  bool setupDirectOneWireLine();
9  bool setupINA260();
10 bool setupLIS3DH();
11 void initSDLogging();
12
13 // ISRs must be declared here so they can be passed to attachInterrupt()
14 void ARDUINO_ISR_ATTR onOvercurrentIRQ();
15 void ARDUINO_ISR_ATTR onImpactIRQ();

```

Setup Implementation (Setup.cpp)

```

1  #include "Setup.h"
2
3  // ISRs: trigger power cut immediately for minimal response time.
4  // ARDUINO_ISR_ATTR places the function in IRAM (fast memory) on ESP32.
5  // digitalWrite is safe to call from ISR; sets the flag for main loop
6  // verification.
7  void ARDUINO_ISR_ATTR onOvercurrentIRQ() {
8      overcurrent_irq = true;
9      if (Config::ENABLE_POWER_CUT_ON_OVERCURRENT) {
10         safety.power_cut_flag = true;
11         digitalWrite(Config::PIN_POWER_CUT, LOW);
12     }
13 }
14 void ARDUINO_ISR_ATTR onImpactIRQ() {
15     impact_irq = true;
16     if (Config::ENABLE_POWER_CUT_ON_CONFIRMED_SHOCK) {
17         safety.power_cut_flag = true;
18         digitalWrite(Config::PIN_POWER_CUT, LOW);
19     }
20 }
21 bool setupHumiditySensors() {
22     bool ok = true;
23
24     // SHT41 on bus 0 (Wire = default I2C)
25     if (!sht41_bus0.begin(Wire)) {
26         Serial.println("[SETUP] SHT41 bus0 not found"); ok = false;
27     } else { Serial.println("[SETUP] SHT41 bus0 OK"); }
28
29     // SHT41 on bus 1 (Wire1 = pins 9/10)
30     if (!sht41_bus1.begin(Wire1)) {
31         Serial.println("[SETUP] SHT41 bus1 not found"); ok = false;
32     } else { Serial.println("[SETUP] SHT41 bus1 OK"); }
33 }

```

```

34 // Si7021 on bus 1 (constructed with &Wire1 in DataTypes.cpp)
35 if (!si7021_bus1.begin()) {
36     Serial.println("[SETUP]_Si7021_bus1_not_found"); ok = false;
37 } else { Serial.println("[SETUP]_Si7021_bus1_OK"); }
38
39 return ok;
40 }
41
42 bool setupDS2484Line(DS2484Line& line) {
43     Serial.print("[SETUP]_Setting_up_"); Serial.println(line.name);
44
45     // DS2484 is an I2C-to-1-Wire bridge chip
46     if (!line.bridge->begin(line.wire, DS248X_ADDRESS)) {
47         Serial.println("[SETUP]_DS2484_not_found");
48         line.present = false; return false;
49     }
50
51     // Check the 1-Wire bus is actually connected and not shorted
52     if (!line.bridge->OneWireReset()) {
53         Serial.println("[SETUP]_1-Wire_reset_failed");
54         if (line.bridge->shortDetected()) Serial.println("[SETUP]_>_Short_detected");
55         if (!line.bridge->presencePulseDetected()) Serial.println("[SETUP]_>_No_presence_pulse");
56         line.present = false; return false;
57     }
58
59     // Walk the bus and collect ROMs of any DS18B20s found
60     line.bridge->OneWireSearchReset();
61     line.found = 0;
62     uint8_t rom[8];
63     while (line.found < Config::MAX_SENSORS_PER_LINE && line.bridge->OneWireSearch(rom)) {
64         if (rom[0] == DS18B20_FAMILY_CODE) { // 0x28 = DS18B20, skip other devices
65             memcpy(line.roms[line.found], rom, 8);
66             Serial.print("[SETUP]_DS18B20_#"); Serial.print(line.found + 1);
67             Serial.print("_ROM:"); printROM(rom); Serial.println();
68             line.found++;
69         }
70     }
71
72     line.present = true;
73     Serial.print("[SETUP]_Total_sensors_on_"); Serial.print(line.name);
74     Serial.print("_="); Serial.println(line.found);
75     return true;
76 }
77
78 bool setupDirectOneWireLine() {
79     Serial.println("[SETUP]_Setting_up_direct_1-Wire_line");
80     directTemps.begin();
81     directTemps.setResolution(9); // 9-bit = 0.5 C , needs 94ms for
        conversion, 12-bit = 0.0625 C but needs 750ms, we want faster updates for
        the impact line
82     directTemps.setWaitForConversion(false); // non-blocking: we manage the 750ms
        ourselves
83
84     line_direct.found = 0;
85     uint8_t count = directTemps.getDeviceCount();
86     for (uint8_t i = 0; i < count && line_direct.found < Config::
        MAX_SENSORS_PER_LINE; i++) {
87         DeviceAddress addr;
88         if (directTemps.getAddress(addr, i)) {
89             memcpy(line_direct.addresses[line_direct.found], addr, 8);

```

```

90     Serial.print("[SETUP]_Direct_DS18B20_#"); Serial.print(line_direct.found
    + 1);
91     Serial.print("_ROM:_"); printDeviceAddress(addr); Serial.println();
92     line_direct.found++;
93 }
94 }
95
96 line_direct.present = (line_direct.found > 0);
97 return line_direct.present;
98 }
99
100 bool setupINA260() {
101     if (!ina260.begin(Config::ADDR_INA260, &Wire)) {
102         Serial.println("[SETUP]_INA260_not_found"); return false;
103     }
104
105     // Configure hardware alert: fires when current > MAX_CURRENT_MA
106     ina260.setAlertType(INA260_ALERT_OVERCURRENT);
107     ina260.setAlertLimit(Config::MAX_CURRENT_MA);
108     ina260.setAlertLatch(INA260_ALERT_LATCH_ENABLED); // stays asserted
    until we read the register
109     ina260.setAlertPolarity(INA260_ALERT_POLARITY_INVERTED); // active-low ->
    RISING edge on MCU pin
110
111     // Connect the alert pin to our ISR
112     pinMode(Config::PIN_INT_CURRENT, INPUT_PULLUP);
113     attachInterrupt(digitalPinToInterrupt(Config::PIN_INT_CURRENT),
        onOvercurrentIRQ, RISING);
114
115     Serial.println("[SETUP]_INA260_OK+_alert_configured");
116     return true;
117 }
118
119 // Write raw registers to configure LIS3DH click/interrupt detection.
120 // The Adafruit driver doesn't expose these, so we write them directly.
121 static void configureImpactInterrupt() {
122     writeIMUReg(0x20, Config::ODR_BITS_200HZ | 0x07); // CTRL_REG1: 200Hz, enable
    X/Y/Z axes
123     writeIMUReg(0x21, 0x00); // CTRL_REG2: no high-pass
    filter
124     writeIMUReg(0x22, 0x40); // CTRL_REG3: route IA1
    event to INT1 pin
125     writeIMUReg(0x23, 0x80 | 0x08 | Config::RANGE_BITS_16G); // CTRL_REG4: block
    update, 16g
126     writeIMUReg(0x24, 0x08); // CTRL_REG5: latch
    interrupt (stays until read)
127     writeIMUReg(0x25, 0x00); // CTRL_REG6: no second
    interrupt config
128     writeIMUReg(0x32, thresholdGToReg(Config::IMPACT_THRESHOLD_G, 16)); //
    INT1_THS: threshold
129     writeIMUReg(0x33, Config::IMPACT_DURATION_SAMPLES); //
    INT1_DURATION: min samples
130     writeIMUReg(0x30, 0x2A); // INT1_CFG: trigger on
    high X, Y, or Z
131     (void)readIMUReg(0x31); // INT1_SRC: dummy read to
    clear any latched flag
132 }
133
134 bool setupLIS3DH() {
135     if (!imu.begin(Config::ADDR_LIS3DH)) {
136         Serial.println("[SETUP]_LIS3DH_not_found"); return false;
137     }
138

```

```

139     imu.setRange(LIS3DH_RANGE_16_G);
140     delay(10); // let the range change settle before writing interrupt registers
141     configureImpactInterrupt();
142
143     pinMode(Config::PIN_INT_IMU, INPUT_PULLUP);
144     attachInterrupt(digitalPinToInterrupt(Config::PIN_INT_IMU), onImpactIRQ,
145                     RISING);
146
147     Serial.println("[SETUP] LIS3DH OK + impact interrupt configured");
148     return true;
149 }
150
151 void initSDLogging() {
152     if (!Config::ENABLE_SD_LOGGING) return;
153     // TODO: SD.begin(Config::PIN_SD_CS) and open/create CSV file
154 }

```

Utilities (Utils.h)

```

1  #pragma once
2  #include "DataTypes.h"
3
4  // Printing helpers
5  void    printROM(const uint8_t* rom); // print 8-byte 1-Wire ROM as
6         hex
7  void    printDeviceAddress(const DeviceAddress addr);
8
9  // Threshold checks      return true if value is valid AND over the limit
10 bool    aboveTempLimit(float value);
11 bool    aboveHumidityLimit(float value);
12
13 // Convert g threshold to LIS3DH register byte (depends on chosen range )
14 uint8_t thresholdGToReg(float g, int range_g);
15
16 // Direct register read/write for LIS3DH (used for interrupt config)
17 void    writeIMUReg(uint8_t reg, uint8_t val);
18 uint8_t readIMUReg(uint8_t reg);
19
20 // Sensor mapping lookup
21 bool    getSensorTempLimit(const uint8_t* address, float& out_limit);
22 bool    isSensorByAddress(const uint8_t* address, Config::SensorPurpose purpose)
23         ;
24
25 // Cut power and log why; sets safety.power_cut_flag
26 void    requestPowerCut(const char* reason);

```

Utilities Implementation (Utils.cpp)

```

1  #include "Utils.h"
2
3  // Print a 1-Wire ROM as 16 hex chars, e.g. "28FF3A1200000032"
4  void printROM(const uint8_t* rom) {
5      for (uint8_t i = 0; i < 8; i++) {
6          if (rom[i] < 16) Serial.print('0'); // zero-pad single hex digits
7          Serial.print(rom[i], HEX);
8      }
9  }
10
11 void printDeviceAddress(const DeviceAddress addr) {

```

```

12   for (uint8_t i = 0; i < 8; i++) {
13       if (addr[i] < 16) Serial.print('0');
14       Serial.print(addr[i], HEX);
15   }
16 }
17
18 // Safe float printer for CSV: avoids printing "-nan" or garbage
19 void printFloatCSV(float value, uint8_t decimals) {
20     if (isnan(value)) Serial.print("nan");
21     else                Serial.print(value, decimals);
22 }
23
24 bool aboveTempLimit(float value) {
25     // isnan check first      NAN comparisons always return false, which would be a
26     // silent bug
27     return !isnan(value) && value > Config::MAX_TEMP_C;
28 }
29
30 bool aboveHumidityLimit(float value) {
31     return !isnan(value) && value > Config::MAX_RH_PERCENT;
32 }
33
34 // LIS3DH stores thresholds as integer LSBs, not raw g.
35 // LSB size depends on the chosen range ( 2g ,  4g ,  8g , 16g ).
36 uint8_t thresholdGToReg(float g, int range_g) {
37     float mg      = g * 1000.0f; // convert to milli-g
38     float mg_per_lsb = 186.0f;    // default: 16g
39     switch (range_g) {
40         case 2: mg_per_lsb = 16.0f; break;
41         case 4: mg_per_lsb = 32.0f; break;
42         case 8: mg_per_lsb = 62.0f; break;
43         case 16: mg_per_lsb = 186.0f; break;
44     }
45     int reg = (int)round(mg / mg_per_lsb);
46     if (reg < 0) reg = 0;
47     if (reg > 127) reg = 127; // register is 7-bit
48     return (uint8_t)reg;
49 }
50
51 // Raw I2C write to a LIS3DH register (the Adafruit driver doesn't expose all
52 // registers)
53 void writeIMUReg(uint8_t reg, uint8_t val) {
54     Wire.beginTransmission(Config::ADDR_LIS3DH);
55     Wire.write(reg);
56     Wire.write(val);
57     Wire.endTransmission();
58 }
59
60 uint8_t readIMUReg(uint8_t reg) {
61     Wire.beginTransmission(Config::ADDR_LIS3DH);
62     Wire.write(reg);
63     Wire.endTransmission(false); // repeated start      keep bus busy between write
64     // and read
65     Wire.requestFrom((int)Config::ADDR_LIS3DH, 1);
66     if (Wire.available()) return Wire.read();
67     return 0;
68 }
69
70 // Pull PIN_POWER_CUT LOW to open the load switch, then log the cause
71 void requestPowerCut(const char* reason) {
72     safety.power_cut_flag = true;
73     digitalWrite(Config::PIN_POWER_CUT, LOW);
74     Serial.print("[ALERT] ⚠ power cut requested due to ");

```

```

72     Serial.println(reason);
73 }
74
75 // Lookup sensor temperature limit by ROM address
76 // Returns true if address is found in the mapping; out_limit is set to the
    limit
77 // Returns false if not found; out_limit is set to Config::MAX_TEMP_C (fallback)
78 bool getSensorTempLimit(const uint8_t* address, float& out_limit) {
79     // Check bus 0
80     for (uint8_t i = 0; i < Config::bus0_sensors_count; i++) {
81         if (memcmp(address, Config::bus0_sensors[i].address, 8) == 0) {
82             out_limit = Config::bus0_sensors[i].temp_limit_c;
83             return true;
84         }
85     }
86     // Check bus 1
87     for (uint8_t i = 0; i < Config::bus1_sensors_count; i++) {
88         if (memcmp(address, Config::bus1_sensors[i].address, 8) == 0) {
89             out_limit = Config::bus1_sensors[i].temp_limit_c;
90             return true;
91         }
92     }
93     // Check direct line
94     for (uint8_t i = 0; i < Config::direct_sensors_count; i++) {
95         if (memcmp(address, Config::direct_sensors[i].address, 8) == 0) {
96             out_limit = Config::direct_sensors[i].temp_limit_c;
97             return true;
98         }
99     }
100     // Not found      use default
101     out_limit = Config::MAX_TEMP_C;
102     return false;
103 }
104
105 // Check if an address matches a specific sensor purpose (e.g., BATTERY, SOCKET)
106 bool isSensorByAddress(const uint8_t* address, Config::SensorPurpose purpose) {
107     // Check bus 0
108     for (uint8_t i = 0; i < Config::bus0_sensors_count; i++) {
109         if (memcmp(address, Config::bus0_sensors[i].address, 8) == 0 &&
110             Config::bus0_sensors[i].purpose == purpose) {
111             return true;
112         }
113     }
114     // Check bus 1
115     for (uint8_t i = 0; i < Config::bus1_sensors_count; i++) {
116         if (memcmp(address, Config::bus1_sensors[i].address, 8) == 0 &&
117             Config::bus1_sensors[i].purpose == purpose) {
118             return true;
119         }
120     }
121     // Check direct line
122     for (uint8_t i = 0; i < Config::direct_sensors_count; i++) {
123         if (memcmp(address, Config::direct_sensors[i].address, 8) == 0 &&
124             Config::direct_sensors[i].purpose == purpose) {
125             return true;
126         }
127     }
128     return false;
129 }

```

GUI - Live Visualization (serial_dashboard.py)


```

1 import serial
2 import threading
3 import collections
4 import pandas as pd
5 from dash import Dash, dcc, html
6 from dash.dependencies import Input, Output, ALL, State
7 from dash.exceptions import PreventUpdate
8 import plotly.graph_objs as go
9 from plotly.subplots import make_subplots
10 import logging
11
12
13 SERIAL_PORT = '/dev/cu.usbmodem101'
14 SERIAL_PORT_2 = '/dev/cu.usbmodem1101'
15 BAUD_RATE = 115200
16 MAX_POINTS = 300
17 UPDATE_SPEED_MS = 500
18
19
20 # columns of the csv from the serial monitor
21 COLUMNS = [
22     "t_ms", "hand_temp_c", "hand_rh", "wrist_temp_c", "wrist_rh", "socket_temp_c",
23     "socket_rh", "bus0_ds1_c", "bus0_ds2_c", "bus0_ds3_c", "bus1_ds1_c",
24     "bus1_ds2_c", "bus1_ds3_c", "socket", "battery", "direct_ds1_c",
25     "imu_ax_g", "imu_ay_g", "imu_az_g", "imu_mag_g", "current_ma", "bus_mv",
26     "power_mw", "temp_limit_flag", "humidity_limit_flag", "overcurrent_limit_flag",
27     "shock_flag", "power_cut_flag", "sensors_ok"
28 ]
29
30
31 # names for printing the same CSV columns
32 NAMES = {
33     "t_ms": "Time[s]",
34     "hand_temp_c": "HandAmbientTemp[ C ]",
35     "hand_rh": "HandRH(%)",
36     "wrist_temp_c": "WristAmbientTemp[ C ]",
37     "wrist_rh": "WristRH(%)",
38     "socket_temp_c": "SocketAmbientTemp[ C ]",
39     "socket_rh": "SocketRH(%)",
40     "bus0_ds1_c": "Motor1Hand,Bus1Temp[ C ]",
41     "bus0_ds2_c": "Motor2Hand,Bus1Temp[ C ]",
42     "bus0_ds3_c": "Motor3Hand,Bus1Temp[ C ]",
43     "bus1_ds1_c": "Motor1Hand,Bus2Temp[ C ]",
44     "bus1_ds2_c": "Motor2Hand,Bus2Temp[ C ]",
45     "bus1_ds3_c": "Motor3Hand,Bus2Temp[ C ]",
46     "direct_ds1_c": "Motor1Wrist,DirectTemp[ C ]",
47     "socket": "SocketContactTemp[ C ]",
48     "battery": "BatteryTemp[ C ]",
49     "imu_ax_g": "IMUAccelX[g]",
50     "imu_ay_g": "IMUAccelY[g]",
51     "imu_az_g": "IMUAccelZ[g]",
52     "imu_mag_g": "IMUMag[g]",
53     "current_ma": "Current[mA]",
54     "bus_mv": "Voltage(V)",
55     "power_mw": "Power[mW]",
56 }
57
58
59 # plot groups
60 GROUPS = {
61     "MotorTemperatures( C )": ["bus0_ds1_c", "bus0_ds2_c", "bus0_ds3_c", "
        bus1_ds1_c", "bus1_ds2_c", "bus1_ds3_c", "direct_ds1_c", "socket", "battery"
        ],

```

```

62     "Body_Temperatures_( C )": ["hand_temp_c", "wrist_temp_c", "socket_temp_c"],
63     "Relative_Humidity(%)": ["hand_rh", "wrist_rh", "socket_rh"],
64     "IMU_Data(g)": ["imu_ax_g", "imu_ay_g", "imu_az_g", "imu_mag_g"],
65     "Power_Data": ["current_ma", "bus_mv", "power_mw"]
66 }
67
68
69 # flags for when the interrupt was triggered, to be plotted
70 FLAGS = [
71     "temp_limit_flag", "humidity_limit_flag", "overcurrent_limit_flag",
72     "shock_flag", "power_cut_flag", "sensors_ok"
73 ]
74
75 # Friendly display names for flags
76 FLAG_NAMES = {
77     "temp_limit_flag": "Temperature_Limit",
78     "humidity_limit_flag": "Humidity_Limit",
79     "overcurrent_limit_flag": "Overcurrent_Limit",
80     "shock_flag": "Shock_Limit",
81     "power_cut_flag": "Power_Cut",
82     "sensors_ok": "Sensors_OK"
83 }
84
85
86 data_queue = collections.deque(maxlen=MAX_POINTS)
87 initial_health_check_done = False # Flag to only check sensor health once at
    startup
88 def is_sensor_healthy(value, sensor_type):
89     """
90     Check if a sensor value is valid based on sensor type.
91     Returns True if healthy, False if broken/invalid.
92     """
93     # Check for None or NaN
94     if value is None or (isinstance(value, float) and pd.isna(value)):
95         return False
96
97
98     if sensor_type == "humidity":
99         # Humidity should not be 0.0 (sensor issue)
100         return value != 0.0
101     elif sensor_type == "temperature":
102         # Temperature should be >= -10 C (reasonable minimum)
103         return value >= -10.0
104     elif sensor_type == "imu":
105         # IMU readings can be any non-zero value, just check for NaN/None
106         return True
107     elif sensor_type == "power":
108         # Power readings should be positive if something is connected
109         return abs(value) > 0
110     elif sensor_type == "current":
111         # Current readings should be non-negative
112         return abs(value) >= 3.0 # resolution of ~1.2 mA, so it sometimes reads
            small values without anything connected to it
113     elif sensor_type == "voltage":
114         # Voltage readings should be non-zero if powered
115         return value != 0.0
116
117
118     return True
119
120
121 def check_sensor_status(df):
122     """

```

```

123     """Check which sensors are healthy based on latest data.
124     Returns a dict mapping variable names to their health status.
125     """
126     if df.empty:
127         return {}
128
129
130     latest = df.iloc[-1]
131     status = {}
132
133
134     # Temperature sensors (Body)
135     for temp_col in ["hand_temp_c", "wrist_temp_c", "socket_temp_c"]:
136         status[temp_col] = is_sensor_healthy(latest[temp_col], "temperature")
137
138
139     # Humidity sensors
140     for rh_col in ["hand_rh", "wrist_rh", "socket_rh"]:
141         status[rh_col] = is_sensor_healthy(latest[rh_col], "humidity")
142
143
144     # Motor temperature sensors
145     for motor_col in ["bus0_ds1_c", "bus0_ds2_c", "bus0_ds3_c", "bus1_ds1_c", "
146         bus1_ds2_c", "bus1_ds3_c", "direct_ds1_c", "socket", "battery"]:
147         status[motor_col] = is_sensor_healthy(latest[motor_col], "temperature")
148
149
150     # IMU sensors
151     for imu_col in ["imu_ax_g", "imu_ay_g", "imu_az_g", "imu_mag_g"]:
152         status[imu_col] = is_sensor_healthy(latest[imu_col], "imu")
153
154
155     # Power, current and voltage sensors
156     status["current_ma"] = is_sensor_healthy(latest["current_ma"], "current")
157     status["bus_mv"] = is_sensor_healthy(latest["bus_mv"], "voltage")
158     status["power_mw"] = is_sensor_healthy(latest["power_mw"], "power")
159
160     return status
161
162
163 # read serial data and add to queue in seperate thread, not to block the
164 # dashboard updates
165 def read_serial_data():
166     import time
167
168     while True:
169         try:
170             try:
171                 ser = serial.Serial(SERIAL_PORT, BAUD_RATE)
172                 print(f"Connected to {SERIAL_PORT}")
173             except:
174                 ser = serial.Serial(SERIAL_PORT_2, BAUD_RATE)
175                 print(f"Connected to {SERIAL_PORT_2}")
176             data_queue.clear() # Clear old data on reconnection
177             print("Data queue cleared - ready for new data")
178
179
180             while True:
181                 line = ser.readline().decode('utf-8').strip()
182
183

```

```

184         # Skip setup messages and empty lines
185         if line.startswith('#') or not line:
186             continue
187
188
189         vals = line.split(',')
190         if len(vals) == len(COLUMNS) and vals[0] != "t_ms":
191             try:
192                 parsed = [float(v) if v.lower() != 'nan' else None for v
193                           in vals]
194                 data_queue.append(parsed)
195                 print(data_queue[-1])
196             except ValueError:
197                 pass
198         except serial.SerialException as e:
199             print(f"Serial connection error: {e}")
200             print("Attempting to reconnect in 2 seconds...")
201             time.sleep(2)
202         except Exception as e:
203             print(f"Unexpected error: {e}")
204             time.sleep(2)
205
206 thread = threading.Thread(target=read_serial_data, daemon=True)
207 thread.start()
208
209
210 # Initialize the app
211 app = Dash(__name__)
212
213
214 # Create control elements for each group
215 # see [https://dash.plotly.com/tutorial](https://dash.plotly.com/tutorial) for
216 # basic informations on plotly (Dash)
217 control_elements = []
218 for group_name, variables in GROUPS.items():
219     group_div = html.Div([
220         html.H5(group_name, style={'font-family': 'Arial', 'marginBottom': '10px'}),
221         dcc.Checklist(
222             id={'type': 'group-checklist', 'index': group_name},
223             options=[{'label': ' ' + NAMES.get(var, var), 'value': var} for var
224                     in variables],
225             value=variables, # All selected by default
226             style={'font-family': 'Arial'},
227             labelStyle={'display': 'block', 'marginBottom': '5px'}
228         ), style={
229             'backgroundColor': '#f5f5f5', 'padding': '10px', 'borderRadius': '3px'
230         })
231     control_elements.append(group_div)
232
233
234 app.layout = html.Div([
235     html.H2("Prosthetic Sensor Live Telemetry", style={'font-family': 'Arial',
236               'text-align': 'center'}),
237
238     # LED Flags Container (Top of the page)
239     html.Div(id='led-container', style={
240         'display': 'flex', 'justify-content': 'center', 'flex-wrap': 'wrap',
241         'gap': '20px', 'marginBottom': '20px', 'font-family': 'Arial'

```

```

242     }),
243
244
245     dcc.Graph(id='live-graph', style={'height': '1200px'}), # Taller graph to
        fit 5 subplots
246     # Control Panel for selecting variables (placed below plots)
247     html.Div([
248         html.H4("Select Data to Display", style={'font-family': 'Arial', '
        marginBottom': '15px'}),
249         html.Div(control_elements, style={
250             'display': 'grid', 'gridTemplateColumns': 'repeat(auto-fit, minmax
        (200px, 1fr))',
251             'gap': '15px', 'marginBottom': '20px'
252         })
253     ], style={
254         'backgroundColor': 'white', 'padding': '15px', 'borderRadius': '5px',
255         'marginTop': '10px', 'marginBottom': '20px', 'boxShadow': '0 2px 4px
        rgba(0,0,0,0.1)'
256     }),
257
258     dcc.Interval(id='graph-update', interval=UPDATE_SPEED_MS, n_intervals=0)
259 ], style={'backgroundColor': '#f8f9fa', 'padding': '20px'})
260
261
262
263 # Helper function to design the LED lights
264 def create_led(name, state):
265     # Default fault flags: Red if 1, Grey if 0
266     color = "#ff3333" if state == 1 else "#e0e0e0"
267
268     # Special case for "sensors_ok": Green if 1, Red if 0
269     if name == "sensors_ok":
270         color = "#33cc33" if state == 1 else "#ff3333"
271
272     display_name = FLAG_NAMES.get(name, name)
273
274     return html.Div([
275         html.Div(style={
276             'width': '24px', 'height': '24px', 'borderRadius': '50%',
277             'backgroundColor': color, 'margin': '0 auto',
278             'boxShadow': f'0 0 10px {color}' if state == 1 else 'none'
279         }),
280         html.Div(display_name, style={'fontSize': '12px', 'marginTop': '5px', '
        fontWeight': 'bold'})
281     ], style={'textAlign': 'center', 'width': '120px'})
282
283
284
285 # Update checklist values to keep all sensors selected
286 @app.callback(
287     Output({'type': 'group-checklist', 'index': ALL}, 'value'),
288     [Input('graph-update', 'n_intervals')],
289     [State({'type': 'group-checklist', 'index': ALL}, 'id')],
290     prevent_initial_call=True
291 )
292 def update_checklist_values(n, ids):
293     global initial_health_check_done
294
295     # Only do this check once at startup when data first arrives
296     if initial_health_check_done or not data_queue or n == 0:
297         raise PreventUpdate
298
299     initial_health_check_done = True

```

```

300
301 df = pd.DataFrame(list(data_queue), columns=COLUMNS)
302 sensor_status = check_sensor_status(df)
303
304 updated_values = []
305 for id_obj in ids:
306     group_name = id_obj['index']
307     if group_name in GROUPS:
308         # Only keep healthy sensors in this group
309         healthy_in_group = [var for var in GROUPS[group_name] if
310                             sensor_status.get(var, True)]
311         updated_values.append(healthy_in_group)
312     else:
313         updated_values.append([])
314
315 return updated_values
316
317 @app.callback(
318     [Output('live-graph', 'figure'),
319      Output('led-container', 'children')],
320     [Input('graph-update', 'n_intervals'),
321      Input({'type': 'group-checklist', 'index': ALL}, 'value')],
322     prevent_initial_call=False
323 )
324 def update_dashboard(n, selected_lists):
325     if not data_queue:
326         return go.Figure(), [create_led(f, 0) for f in FLAGS]
327
328     # Flatten the selected variables from all groups
329     selected_variables = []
330     if selected_lists:
331         for var_list in selected_lists:
332             if var_list:
333                 selected_variables.extend(var_list)
334
335     df = pd.DataFrame(list(data_queue), columns=COLUMNS)
336
337     # 1. Update the LED lights based on the single most recent data row
338     latest_data = df.iloc[-1]
339     led_elements = [create_led(flag, latest_data[flag]) for flag in FLAGS]
340
341     # 2. Update the 5 Subplots - respect user's manual selections
342     fig = make_subplots(
343         rows=5, cols=1,
344         shared_xaxes=True,
345         vertical_spacing=0.03,
346         subplot_titles=list(GROUPS.keys())
347     )
348
349     row_idx = 1
350     for title, variables in GROUPS.items():
351         for col in variables:
352             # Only add trace if it's selected by user (no health filtering after
353             startup)
354             if col in selected_variables:
355                 y_data = df[col]
356                 # Convert voltage from mV to V

```

```

361         if col == 'bus_mv':
362             y_data = y_data / 1000
363         fig.add_trace(
364             go.Scattergl(
365                 x=df['t_ms'] / 1000,
366                 y=y_data,
367                 mode='lines',
368                 name=NAMES.get(col, col)
369             ),
370             row=row_idx, col=1
371         )
372
373         # Add invisible dummy trace to keep axes visible even when empty
374         fig.add_trace(
375             go.Scattergl(
376                 x=[],
377                 y=[],
378                 mode='lines',
379                 showlegend=False,
380                 hoverinfo='skip'
381             ),
382             row=row_idx, col=1
383         )
384         row_idx += 1
385
386     fig.update_layout(
387         template="plotly_white",
388         hovermode="x_unified",
389         margin=dict(l=40, r=40, t=40, b=40),
390         legend=dict(orientation="v", yanchor="top", y=1, xanchor="left", x=1.02)
391     )
392
393     # Ensure axes sync when zooming in on time
394     fig.update_xaxes(matches='x')
395
396     # Fix y-axis ranges for specific subplot types
397     # Temp should normally range from 0 to 80, so for this, no need to plot much
398     more
399     # fig.update_yaxes(range=[0, 90], row=1, col=1)
400     # fig.update_yaxes(range=[0, 90], row=2, col=1)
401     # RH plotted on full range (0 to 100%)
402     # fig.update_yaxes(range=[0, 100], row=3, col=1)
403
404     # Ensure axes and grid are always visible, even when empty
405     fig.update_xaxes(showgrid=True, showline=True, zeroline=False)
406     fig.update_yaxes(showgrid=True, showline=True, zeroline=False)
407
408     # Add x-axis label to bottom subplot
409     fig.update_xaxes(title_text="Time[s]", row=5, col=1)
410
411     return fig, led_elements
412
413 if __name__ == '__main__':
414     # Suppress Werkzeug debug logs to see serial prints cleanly
415     logging.getLogger('werkzeug').setLevel(logging.ERROR)
416
417
418
419
420
421
422

```

```
423 |  
424 |     print("Starting server... Open [http://127.0.0.1:8050] (http  
425 |           ://127.0.0.1:8050) in your browser.")  
    |     app.run(debug=False)
```